

[Nombre de la universidad]

[Facultad · Carrera]

Base de Datos III · Proyecto: Implementación de un Data Warehouse

PROYECTO DE DATA WAREHOUSE

Data Warehouse del Campeonato Mundial de Fórmula 1

DataMart de Rendimiento en Carrera · Temporadas 1950–2024

Primera presentación · Puntos 1 a 5

Docente	Rodnie Montaña Aguilera
Integrantes	Jhonatan Moisés Villca [Integrante 2] [Integrante 3]
Semestre	II/2026
Fecha	Octubre de 2026

Índice

Introducción	3
1. Tema del proyecto	4
1.1 Tema seleccionado	4
1.2 Conjunto de datos	4
1.3 Contexto de los datos	5
1.4 Necesidad de análisis	6
1.5 Justificación frente a temáticas comunes	6
1.6 Observaciones iniciales de calidad de datos	6
2. Objetivos del Data Warehouse	8
2.1 Objetivo general	8
2.2 Objetivos específicos	8
2.3 Indicadores asociados a cada objetivo	8
3. Modelado conceptual del DataMart	10
3.1 Procesos de negocio identificados	10
3.2 DataMarts propuestos	10
3.3 Matriz de bus y dimensiones conformadas	11
3.4 Evaluación y selección del DataMart	11
3.5 Modelo conceptual del DataMart seleccionado	12
3.6 Granularidad del hecho	13
3.7 Jerarquías de las dimensiones	13
4. Modelado lógico del DataMart	15
4.1 Esquema en estrella	15
4.2 Tablas de dimensiones	17
4.3 Tabla de hechos	19
4.4 Claves y estrategia de dimensiones	21
4.5 Relaciones entre tablas	21
4.6 Métricas y su clasificación	22
4.7 Verificación del modelo frente a los objetivos	22
5. Modelado físico del DataMart	24
5.1 Plataforma y convenciones	24
5.2 Tipos de datos	24
5.3 Restricciones e integridad referencial	24
5.4 Índices	26
5.5 Scripts de implementación	27
5.6 Evidencia de la implementación	27
5.7 Preparación para la carga ETL	28
Conclusiones y próximos pasos	30
Referencias	31
Anexos: scripts SQL	32

Introducción

Este documento corresponde a la primera presentación del proyecto de la asignatura Base de Datos III, cuyo propósito es implementar un Data Warehouse a partir de un conjunto de datos público. En esta entrega se desarrollan los puntos 1 a 5 de la consigna: la selección del tema, los objetivos del Data Warehouse y los modelos conceptual, lógico y físico del DataMart elegido. Los procesos ETL con Apache Hop, los cubos OLAP, los reportes y los cuadros de mando corresponden a la segunda presentación.

El dominio elegido es el Campeonato Mundial de Fórmula 1. A partir del conjunto de datos histórico publicado en Kaggle, que reúne setenta y cinco temporadas de resultados, se propone un DataMart orientado a medir el rendimiento de pilotos, escuderías y circuitos. El modelo dimensional se implementó en PostgreSQL 16; los scripts completos y su evidencia de ejecución se incluyen en el documento y en los anexos.

Conjunto de datos Formula 1 World Championship 1950–2024 (Kaggle)	DataMart seleccionado Rendimiento en Carrera	Modelo Estrella: 1 hecho y 6 dimensiones
Granularidad Resultado de un piloto con un auto en un Gran Premio	Volumen estimado ≈ 26 700 hechos (1950–2024)	DBMS PostgreSQL 16 · esquema dw_f1

1. Tema del proyecto

1.1 Tema seleccionado

Tema

Análisis histórico del rendimiento competitivo de pilotos, escuderías y circuitos en el Campeonato Mundial de Fórmula 1, temporadas 1950 a 2024.

La Fórmula 1 es la máxima categoría del automovilismo de monoplazas y está regulada por la Federación Internacional del Automóvil (FIA). Cada temporada se compone de una serie de Grandes Premios —siete en 1950 y veinticuatro en 2024— que se disputan en circuitos de distintos países. En cada carrera los pilotos compiten representando a una escudería (constructor) y, según su posición final, reciben puntos que se acumulan en dos campeonatos: el de Pilotos y el de Constructores.

Es un deporte intensivo en datos. Los equipos analizan su rendimiento y confiabilidad para decidir inversiones y estrategias; la prensa especializada compara épocas y pilotos; los patrocinadores evalúan la visibilidad que obtienen (victorias, podios, puntos); y los organizadores necesitan saber qué circuitos ofrecen carreras más disputadas. Todas estas necesidades parten de un mismo hecho: **el resultado que obtiene cada piloto en cada Gran Premio.**

1.2 Conjunto de datos

Ficha del conjunto de datos

Nombre	Formula 1 World Championship (1950 – 2024)
Publicado por	Rohan Rao, en Kaggle
Dirección	https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020
Origen de los datos	Ergast Developer API, base de datos histórica de resultados de la Fórmula 1
Formato	14 archivos CSV relacionados mediante identificadores numéricos (modelo relacional normalizado)
Cobertura	Temporadas 1950 a 2024 (≈ 1 125 Grandes Premios)
Valores nulos	Representados con la cadena \N

Archivos del conjunto de datos y su uso en el proyecto

Archivo	Contenido	Registros (aprox.)	Uso
races.csv	Carreras: temporada, ronda, circuito, nombre, fecha y hora	1 125	dim_carrera, dim_tiempo
results.csv	Resultado de cada piloto en cada carrera: grilla, posición, puntos, vueltas, tiempos, estado	26 700	Tabla de hechos
drivers.csv	Pilotos: nombre, código, número, fecha de nacimiento, nacionalidad	860	dim_piloto
constructors.csv	Escuderías y su nacionalidad	210	dim_escuderia
circuits.csv	Circuitos: nombre, localidad, país, coordenadas y altitud	77	dim_circuito

Archivo	Contenido	Registros (aprox.)	Uso
status.csv	Catálogo de estados de finalización (Finished, Engine, Accident...)	139	dim_estado
driver_standings.csv	Puntos y posición de cada piloto en el campeonato tras cada carrera	34 800	Métricas de campeonato
sprint_results.csv	Resultados de las carreras sprint (desde 2021)	360	dim_carrera (tiene_sprint)
seasons.csv	Temporadas y enlace de referencia	75	Cubierto por dim_tiempo
qualifying.csv	Clasificación: posición y tiempos en Q1, Q2 y Q3	10 500	DataMart 3 (propuesto)
lap_times.csv	Tiempo y posición de cada piloto en cada vuelta (desde 1996)	589 000	DataMart 2 (propuesto)
pit_stops.csv	Paradas en boxes: vuelta y duración (desde 2011)	11 400	DataMart 2 (propuesto)
constructor_results.csv	Puntos de cada escudería en cada carrera	12 600	DataMart 3 (propuesto)
constructor_standings.csv	Puntos y posición de cada escudería en el campeonato tras cada carrera	13 400	DataMart 3 (propuesto)

Las cantidades de registros son aproximadas para la versión 1950–2024 del dataset y se confirmarán durante la extracción del proceso ETL.

1.3 Contexto de los datos

Para interpretar correctamente los datos es necesario conocer los conceptos del dominio que aparecen en los archivos:

Temporada y ronda	Cada año es una temporada del campeonato; la ronda indica el orden de cada Gran Premio dentro de ella (races.year, races.round).
Gran Premio y circuito	Un Gran Premio es una carrera puntuable que se disputa en un circuito. Un mismo circuito puede albergar Grandes Premios con distintos nombres a lo largo de la historia.
Piloto y escudería	El piloto conduce un auto de una escudería. Un piloto puede cambiar de escudería entre temporadas e incluso durante una misma temporada; en los primeros años varios pilotos compartían un mismo auto.
Grilla de salida	Posición desde la que larga el piloto (grid). El valor 0 indica que largó desde los boxes o que no tiene posición asignada.
Resultado	Posición final clasificada (position) u otro código (positionText): R = retirado, D = descalificado, E = excluido, W = retirado antes de largar, F = no clasificó a la carrera, N = no clasificado.
Estado	Motivo con el que terminó la carrera: «Finished», «+1 Lap», «Engine», «Accident», «Collision», etc. (status.csv).
Puntos	Se otorgan según la posición final. El sistema de puntuación cambió varias veces (por ejemplo, 9 puntos por victoria entre 1961 y 1990 y 25 desde 2010), por lo que los puntos de distintas épocas no son directamente comparables.
Campeonato	Suma de puntos de la temporada. driver_standings.csv registra los puntos y la posición acumulados de cada piloto después de cada carrera.

1.4 Necesidad de análisis

Los datos están organizados en un modelo relacional normalizado de catorce tablas, adecuado para registrar resultados pero no para el análisis histórico. Preguntas como «¿qué escudería dominó cada década?», «¿en qué circuitos es más determinante largar desde la pole?» o «¿qué pilotos fueron más efectivos si se comparan épocas con el mismo sistema de puntos?» exigen unir varias tablas, interpretar códigos, convertir tiempos almacenados como texto y tratar distintos sistemas de puntuación en cada consulta.

La necesidad de análisis que el proyecto busca resolver es **evaluar de forma consistente y comparable en el tiempo el rendimiento de pilotos, escuderías y circuitos**: quién gana, con qué frecuencia, con qué confiabilidad y bajo qué condiciones. Los usuarios potenciales son:

- **Equipos (escuderías)**: confiabilidad mecánica, conversión de la posición de salida en resultado y comparación con sus rivales.
- **Analistas deportivos y medios**: comparaciones históricas entre pilotos y épocas, dominio de escuderías y evolución del campeonato.
- **Patrocinadores**: pilotos y escuderías con mayor frecuencia de victorias, podios y puntos.
- **Organizadores de Grandes Premios**: circuitos con más adelantamientos y menor dependencia de la posición de salida.

El Data Warehouse integra y depura estos datos alrededor del resultado de cada carrera, de modo que las preguntas puedan responderse con indicadores, cubos OLAP y cuadros de mando.

1.5 Justificación frente a temáticas comunes

La consigna excluye temáticas repetitivas como supermercados, hoteles, comercio electrónico o sistemas genéricos de ventas. El tema elegido plantea un problema analítico distinto y con complejidad suficiente para un Data Warehouse:

- **No es un proceso de compra y venta**. El hecho central es un resultado deportivo con métricas de naturaleza diversa: ordinales (posiciones), aditivas (puntos, vueltas), semiaditivas (puntos acumulados del campeonato) y temporales (tiempos de vuelta).
- **Profundidad histórica**. Setenta y cinco temporadas con cambios de reglamento obligan a normalizar los puntos para comparar épocas y a manejar datos que solo existen en ciertos periodos.
- **Varios procesos y granularidades**. Resultados, clasificación, vueltas, paradas en boxes y campeonatos permiten proponer tres DataMarts con dimensiones conformadas.
- **Calidad de datos no trivial**. Nulos codificados como texto, tiempos en formato «m:ss.mmm», pilotos con más de un resultado en la misma carrera y medios puntos en carreras acortadas.

1.6 Observaciones iniciales de calidad de datos

La revisión de la estructura del dataset permitió identificar los siguientes aspectos, que se tratarán en el proceso ETL de la segunda presentación y que ya se consideraron en el diseño:

Hallazgos de calidad y tratamiento previsto

Hallazgo	Ejemplo	Tratamiento en el diseño / ETL
Nulos codificados como texto	\N en position, code, time	Conversión a NULL ; columnas que lo admiten declaradas como nulas
Tiempos almacenados como texto	fastestLapTime = "1:27.452"	Conversión a milisegundos (INTEGER) para poder promediar y comparar

Hallazgo	Ejemplo	Tratamiento en el diseño / ETL
Datos que existen solo en ciertas épocas	Vuelta rápida desde 2004, tiempos por vuelta desde 1996, paradas desde 2011	Se mantienen nulos; la cobertura se indica en los reportes
Más de un resultado por piloto en una carrera	Autos compartidos en los años cincuenta	El grano se define por resultado (resultId), no por piloto y carrera
Puntos fraccionarios	Medios puntos en carreras acortadas (p. ej. Bélgica 2021)	Tipo NUMERIC(5,2)
Cambios del sistema de puntuación	9 puntos por victoria (1961–1990) frente a 25 (desde 2010)	Métrica adicional puntos_sistema_actual
Códigos de resultado en texto	R, D, E, W, F, N en positionText	Indicadores es_abandono y es_finalizado
Estados muy detallados	139 estados distintos (Engine, Gearbox, Collision...)	Agrupación en categorías de análisis en dim_estado
500 Millas de Indianápolis (1950–1960)	Pilotos que solo corrieron esa prueba puntuable	Se conservan; se identifican por el nombre del Gran Premio

2. Objetivos del Data Warehouse

2.1 Objetivo general

Diseñar e implementar un Data Warehouse para el Campeonato Mundial de Fórmula 1, comenzando por el DataMart de Rendimiento en Carrera, que integre y depure los resultados históricos de cada Gran Premio (1950–2024) para analizar el rendimiento de pilotos, escuderías y circuitos mediante indicadores, consultas analíticas, cubos OLAP y cuadros de mando que apoyen la toma de decisiones deportivas y comerciales.

2.2 Objetivos específicos

Cada objetivo específico es medible mediante indicadores calculados con las métricas del DataMart (sección 2.3) y define qué hechos, dimensiones y niveles de detalle debe contener el modelo.

Objetivos específicos y preguntas de negocio

Código	Objetivo específico	Pregunta de negocio
OE1	Medir la conversión de la posición de salida en resultado final, calculando el porcentaje de victorias obtenidas desde la pole position y el promedio de posiciones ganadas por temporada, circuito y escudería.	¿Cuán determinante es largar primero? ¿Qué escuderías remontan más?
OE2	Determinar el nivel de dominio de las escuderías por temporada y por década mediante su participación porcentual en las victorias, los podios y los puntos.	¿Qué escudería dominó cada época y con qué margen?
OE3	Cuantificar la confiabilidad de las escuderías mediante la tasa de abandono y la tasa de finalización por temporada y circuito, identificando las categorías de causa más frecuentes.	¿Qué equipos abandonan más y por qué motivo?
OE4	Comparar el rendimiento histórico de los pilotos mediante los puntos por carrera normalizados al sistema actual, las tasas de victoria y de podio y la edad promedio al ganar, y seguir su evolución en el campeonato ronda a ronda.	¿Quiénes fueron los pilotos más efectivos, comparando épocas de forma justa?
OE5	Caracterizar los circuitos y países sede según la cantidad de Grandes Premios disputados, el índice de adelantamiento, la tasa de abandono y el porcentaje de victorias desde la pole.	¿En qué circuitos es más difícil adelantar o más frecuente abandonar?
OE6	Evaluar la evolución del ritmo de carrera por circuito y temporada, desde 2004, mediante el tiempo y la velocidad media de la vuelta más rápida y su variación porcentual interanual.	¿Los autos son cada año más rápidos en cada circuito?

2.3 Indicadores asociados a cada objetivo

Los indicadores (KPI) se calculan a partir de las métricas de la tabla de hechos. Las tasas y porcentajes se recalculan siempre desde sus componentes aditivos (numerador y denominador), nunca promediando porcentajes ya calculados.

Indicadores, fórmulas y métricas utilizadas

OE	Indicador	Fórmula	Tipo
OE1	% de conversión de la pole	$\Sigma(es_victoria \times es_pole) / \Sigma es_pole \times 100$	Porcentaje

OE	Indicador	Fórmula	Tipo
OE1	Promedio de posiciones ganadas	AVG(posiciones_ganadas)	Promedio
OE2	% de victorias en la temporada	$\Sigma \text{ es_victoria (escudería)} / \Sigma \text{ es_victoria (temporada)} \times 100$	Porcentaje
OE2	% de podios en la temporada	$\Sigma \text{ es_podio (escudería)} / \Sigma \text{ es_podio (temporada)} \times 100$	Porcentaje
OE2	Cuota de puntos	$\Sigma \text{ puntos (escudería)} / \Sigma \text{ puntos (temporada)} \times 100$	Porcentaje
OE3	Tasa de abandono	$\Sigma \text{ es_abandono} / \text{N.º participaciones} \times 100$	Porcentaje
OE3	Tasa de finalización	$\Sigma \text{ es_finalizado} / \text{N.º participaciones} \times 100$	Porcentaje
OE3	Distribución de causas de abandono	Abandonos por categoría de estado / total de abandonos $\times 100$	Porcentaje
OE4	Puntos normalizados por carrera	$\Sigma \text{ puntos_sistema_actual} / \text{N.º participaciones}$	Promedio
OE4	Tasa de victorias y de podios	$\Sigma \text{ es_victoria (o es_podio)} / \text{N.º participaciones} \times 100$	Porcentaje
OE4	Edad promedio al ganar	AVG(edad_piloto) donde es_victoria = 1	Promedio
OE4	Evolución en el campeonato	puntos_acumulados_temporada y posicion_campeonato por ronda	Último valor
OE5	Grandes Premios disputados	COUNT(DISTINCT carrera) por circuito	Conteo
OE5	Índice de adelantamiento	AVG(posiciones_ganadas) por circuito	Promedio
OE5	Tasa de abandono y % de victorias desde la pole por circuito	Igual a OE3 y OE1, agrupados por circuito	Porcentaje
OE6	Mejor vuelta del Gran Premio	MIN(tiempo_vuelta_rapida_ms)	Mínimo
OE6	Velocidad media de vuelta rápida	AVG(velocidad_vuelta_rapida_kmh)	Promedio
OE6	Variación interanual del ritmo	$(\text{velocidad año } t - \text{velocidad año } t-1) / \text{velocidad año } t-1 \times 100$	Porcentaje

3. Modelado conceptual del DataMart

3.1 Procesos de negocio identificados

Siguiendo la metodología dimensional de Kimball, primero se identificaron los procesos de negocio que generan eventos medibles en las fuentes de datos:

Procesos de negocio presentes en el conjunto de datos

Proceso	Evento que se registra	Fuente	Cobertura
Resultado de carrera	Posición, puntos, vueltas y estado final de cada piloto en un Gran Premio	results, races	1950–2024
Clasificación	Tiempos de las sesiones Q1, Q2 y Q3 que definen la grilla	qualifying	Completa desde 2003
Vueltas en carrera	Tiempo y posición de cada piloto en cada vuelta	lap_times	1996–2024
Paradas en boxes	Vuelta y duración de cada parada	pit_stops	2011–2024
Campeonato de pilotos	Puntos, posición y victorias acumulados tras cada ronda	driver_standings	1950–2024
Campeonato de constructores	Puntos y posición de cada escudería tras cada ronda	constructor_standings, constructor_results	1958–2024

3.2 DataMarts propuestos

A partir de los procesos anteriores se proponen tres DataMarts, cada uno orientado a un grupo de usuarios y a un tipo de análisis distinto:

DataMart 1 · Rendimiento en Carrera SELECCIONADO	
Procesos	Resultado de carrera; el campeonato de pilotos se incorpora como métrica acumulada.
Hecho y grano	fact_resultado_carrera: un registro por resultado de un piloto con un auto en un Gran Premio (≈ 26 700).
Dimensiones	Tiempo, Carrera, Circuito, Piloto, Escudería, Estado.
Métricas	Puntos, posiciones, posiciones ganadas, vueltas, tiempos, vuelta rápida, edad; indicadores de victoria, podio, pole y abandono; puntos acumulados.
Usuarios	Analistas deportivos, equipos, medios y patrocinadores.
Cobertura	Completa, 1950–2024.

DataMart 2 · Estrategia en Pista	
Procesos	Vueltas en carrera y paradas en boxes.
Hechos y grano	fact_vuelta: un registro por vuelta de un piloto en una carrera (≈ 589 000). fact_parada_boxes: un registro por parada de un piloto en una carrera (≈ 11 400).
Dimensiones	Tiempo, Carrera, Circuito, Piloto, Escudería (derivada del resultado del piloto en esa carrera).
Métricas	Tiempo de vuelta, posición en cada vuelta, duración de la parada, vuelta de la parada.
Usuarios	Ingenieros de estrategia de los equipos.
Limitación	Solo desde 1996 (vueltas) y 2011 (paradas).

DataMart 3 · Clasificación y Campeonato

Procesos	Clasificación y campeonatos de pilotos y de constructores.
Hechos y grano	fact_clasificacion: un registro por piloto en la clasificación de un Gran Premio. fact_posicion_campeonato: instantánea periódica con los puntos y la posición de cada piloto o escudería tras cada ronda.
Dimensiones	Tiempo, Carrera, Circuito, Piloto, Escudería.
Métricas	Tiempos Q1–Q3 y posición de clasificación (no aditivas); puntos y victorias acumulados (semiaditivas).
Usuarios	Medios, aficionados y organizadores.
Limitación	Clasificación completa solo desde 2003.

3.3 Matriz de bus y dimensiones conformadas

La matriz de bus relaciona cada proceso de negocio con las dimensiones que lo describen. Las dimensiones compartidas se diseñan una sola vez (*dimensiones conformadas*) para que los tres DataMarts puedan integrarse en el futuro en un mismo Data Warehouse.

Matriz de bus del Data Warehouse de Fórmula 1

Proceso de negocio	Tiempo	Carrera	Circuito	Piloto	Escudería	Estado	DataMart
Resultado de carrera	✓	✓	✓	✓	✓	✓	1 · Rendimiento en Carrera
Vueltas en carrera	✓	✓	✓	✓	✓*		2 · Estrategia en Pista
Paradas en boxes	✓	✓	✓	✓	✓*		2 · Estrategia en Pista
Clasificación	✓	✓	✓	✓	✓		3 · Clasificación y Campeonato
Campeonato de pilotos	✓	✓	✓	✓			3 (y métrica acumulada en 1)
Campeonato de constructores	✓	✓	✓		✓		3 · Clasificación y Campeonato

* La escudería no está en el archivo de origen; se obtiene del resultado del mismo piloto en la misma carrera.

3.4 Evaluación y selección del DataMart

Para elegir el DataMart a desarrollar se evaluaron las tres propuestas con criterios ponderados (puntaje de 1 a 5):

Matriz de evaluación de los DataMarts propuestos

Criterio	Peso	DM1 Rendimiento	DM2 Estrategia	DM3 Clasificación
Alineación con los objetivos del proyecto	30 %	5	2	3
Cobertura histórica de los datos	25 %	5	2	4
Disponibilidad y calidad de los datos	20 %	4	3	4
Valor para los usuarios del negocio	15 %	5	4	4
Viabilidad del ETL y de los cubos OLAP	10 %	5	3	4
Puntaje ponderado	100 %	4,80	2,60	3,70

Se selecciona el **DataMart 1 · Rendimiento en Carrera** por las siguientes razones:

1. El resultado de carrera es el proceso central del deporte: de él derivan los puntos, las victorias y los campeonatos.
2. Es el único proceso con cobertura completa de 1950 a 2024, lo que permite el análisis histórico entre épocas.
3. Permite responder los seis objetivos específicos planteados.
4. Sus dimensiones (Tiempo, Carrera, Circuito, Piloto, Escudería) son las dimensiones conformadas que reutilizarán los DataMarts 2 y 3.
5. Su volumen es manejable para la carga con Apache Hop y, al mismo tiempo, presenta suficientes problemas de calidad y transformación para justificar el proceso ETL.

3.5 Modelo conceptual del DataMart seleccionado

El proceso de negocio modelado es la **disputa de un Gran Premio**, concretamente el resultado oficial que obtiene cada participante. El hecho *Resultado de carrera* se describe mediante seis dimensiones que responden a las preguntas *cuándo* (Tiempo), *qué evento* (Carrera), *dónde* (Circuito), *quién* (Piloto), *con qué equipo* (Escudería) y *cómo terminó* (Estado).

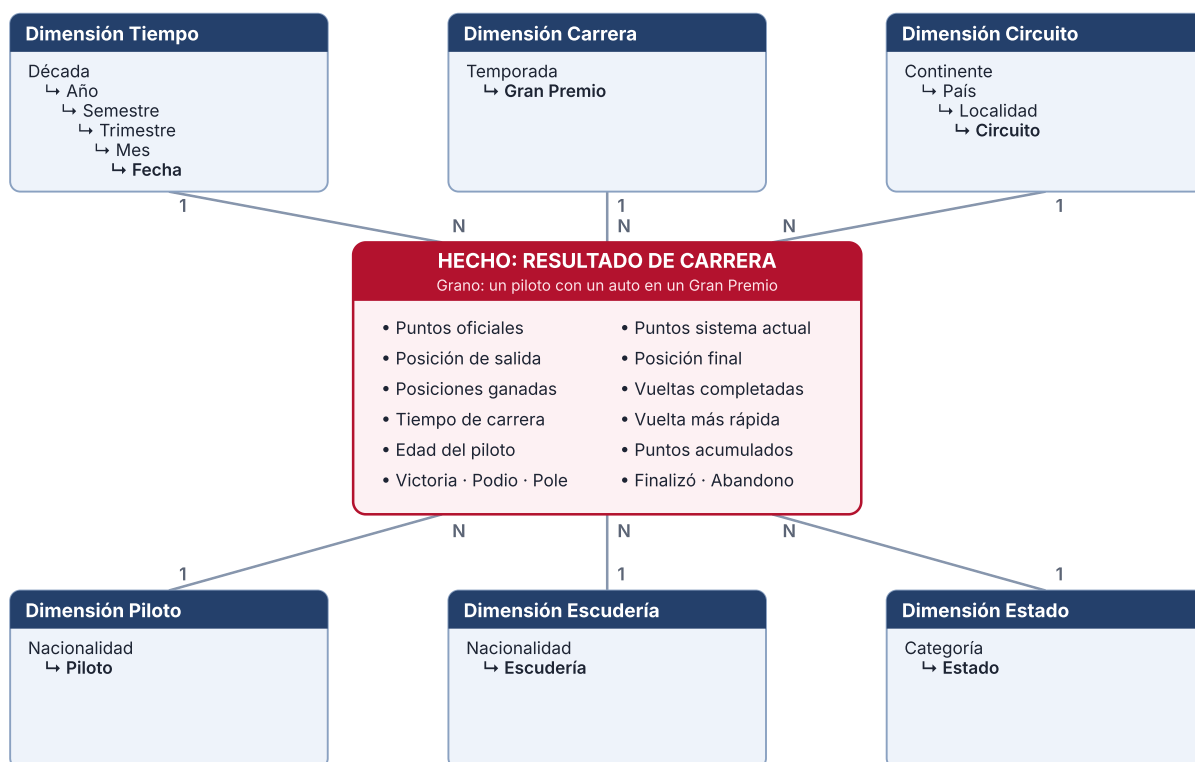


Figura 1. Modelo conceptual del DataMart Rendimiento en Carrera: hecho central, dimensiones y jerarquías.

Elementos del modelo conceptual

Elemento	Tipo	Descripción
Resultado de carrera	Hecho	Desempeño de un piloto en un Gran Premio: posiciones, puntos, vueltas, tiempos e indicadores de victoria, podio, pole y abandono.
Tiempo	Dimensión	Fecha de la carrera con su jerarquía de calendario hasta la década.
Carrera	Dimensión	El Gran Premio: temporada, ronda, nombre, carrera sprint y sistema de puntuación vigente.
Circuito	Dimensión	Lugar de la carrera, con su jerarquía geográfica (continente, país, localidad).
Piloto	Dimensión	Datos personales y deportivos del piloto.
Escudería	Dimensión	Equipo o constructor con el que corrió el piloto.
Estado	Dimensión	Forma en que terminó la carrera, agrupada en categorías (finalizó, falla mecánica, accidente...).

3.6 Granularidad del hecho

Declaración del grano

Cada registro de la tabla de hechos representa **el resultado oficial de un piloto, conduciendo un auto de una escudería, en un Gran Premio del Campeonato Mundial**. Equivale a una fila de `results.csv` (identificada por `resultId`).

Ejemplo de un registro: Max Verstappen · Red Bull · Gran Premio de Baréin 2023 (5 de marzo de 2023): largó 1.º, terminó 1.º, 25 puntos, 57 vueltas, estado «Finished».

- **Es el grano atómico** disponible para el proceso de resultados. A partir de él se puede agregar por temporada, década, escudería, país o categoría sin perder información.
- **No se usa «piloto por carrera»** como grano porque, en las primeras temporadas, un piloto podía compartir autos y tener más de un resultado en el mismo Gran Premio. Por eso la unicidad se garantiza con `id_resultado_origen`.
- **Queda fuera del grano** el detalle por vuelta, las paradas en boxes y la clasificación (DataMarts 2 y 3). Las carreras sprint se identifican en la dimensión Carrera, pero sus resultados no forman parte de este hecho.
- **Tipo de tabla de hechos:** transaccional; cada fila corresponde a un evento. Los puntos acumulados del campeonato se agregan como una métrica semiaditiva del mismo evento.
- **Volumen:** ≈ 26 700 registros para 1950–2024 y un crecimiento aproximado de 480 registros por temporada (24 carreras × 20 pilotos).

3.7 Jerarquías de las dimensiones

Las jerarquías permiten navegar desde niveles agregados hacia el detalle (*drill-down*) y servirán como base para los niveles de los cubos OLAP.

Jerarquías definidas

Dimensión	Jerarquía (de lo general al detalle)	Uso en el análisis
Tiempo	Década → Año → Semestre → Trimestre → Mes → Fecha	Evolución histórica y comparación de épocas
Carrera	Temporada → Gran Premio (ronda)	Rendimiento por temporada y evolución ronda a ronda
Circuito	Continente → País → Localidad → Circuito	Análisis geográfico y efecto del circuito
Piloto	Nacionalidad → Piloto	Rankings de pilotos y de países
Escudería	Nacionalidad → Escudería	Dominio de equipos
Estado	Categoría → Estado	Causas de abandono

4. Modelado lógico del DataMart

4.1 Esquema en estrella

El modelo lógico adopta un **esquema en estrella**: una tabla de hechos central, `fact_resultado_carrera`, rodeada por seis tablas de dimensiones desnormalizadas. Se descartó el esquema copo de nieve —por ejemplo, separar país o nacionalidad en tablas propias— porque las dimensiones son pequeñas (menos de 1 200 filas, salvo Tiempo) y la estrella simplifica las consultas, la carga ETL y la definición de los cubos OLAP de la segunda etapa.

La Figura 2, en la página siguiente, muestra todas las tablas con sus atributos, tipos de datos y claves. Cada dimensión se relaciona con la tabla de hechos mediante una relación uno a muchos (1:N): un miembro de la dimensión puede aparecer en muchos resultados, y cada resultado referencia exactamente un miembro de cada dimensión.

Resumen del modelo lógico

7 tablas · 6 dimensiones con 51 atributos · 1 tabla de hechos con 32 columnas: 1 clave primaria, 6 claves foráneas, 3 dimensiones degeneradas, 14 métricas, 7 indicadores 0/1 y 1 columna de auditoría.



Figura 2. Modelo lógico en estrella del DataMart Rendimiento en Carrera.

PK clave primaria (sustituta) **FK** clave foránea **NK** clave natural (única) **DD** dimensión degenerada ? admite nulos || —< relación 1 : N

4.2 Tablas de dimensiones

A continuación se detalla cada dimensión con sus atributos, tipo de dato, claves, nulabilidad, descripción y origen en el dataset (o regla de transformación del ETL).

Dimensión Tiempo

Calendario diario de 1950 a 2030, generado por SQL antes del ETL. Usa una clave sustituta inteligente (AAAAMMDD), habitual en dimensiones de fecha, que facilita la lectura y la partición por fecha.

Dimensión Tiempo (dim_tiempo)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_tiempo	INTEGER	PK	No	Clave sustituta inteligente con formato AAAAMMDD	Calculada a partir de la fecha
fecha	DATE	NK	No	Fecha calendario	generate_series 1950-01-01 a 2030-12-31
dia	SMALLINT		No	Día del mes (1-31)	EXTRACT(DAY)
dia_semana	SMALLINT		No	Día de la semana ISO (1 = lunes, 7 = domingo)	EXTRACT(ISODOW)
nombre_dia	VARCHAR(10)		No	Nombre del día en español	Derivado
mes	SMALLINT		No	Mes (1-12)	EXTRACT(MONTH)
nombre_mes	VARCHAR(10)		No	Nombre del mes en español	Derivado
trimestre	SMALLINT		No	Trimestre (1-4)	EXTRACT(QUARTER)
semestre	SMALLINT		No	Semestre (1-2)	Derivado
anio	SMALLINT		No	Año calendario (coincide con la temporada)	EXTRACT(YEAR)
decada	SMALLINT		No	Primer año de la década (1950, 1960, ...)	anio - anio % 10
nombre_decada	VARCHAR(15)		No	Etiqueta de la década, p. ej. «Década de 1990»	Derivado

Dimensión Carrera

Un registro por Gran Premio. Conserva el sistema de puntuación vigente para interpretar correctamente los puntos de cada época.

Dimensión Carrera (Gran Premio) (dim_carrera)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_carrera	INTEGER	PK	No	Clave sustituta (IDENTITY)	Generada por el DBMS
id_carrera_origen	INTEGER	NK	No	Identificador de la carrera en la fuente	racess.raceld
temporada	SMALLINT		No	Año de la temporada (única junto con ronda)	racess.year
ronda	SMALLINT		No	Número de la carrera dentro de la temporada	racess.round
total_rondas_temporada	SMALLINT		No	Cantidad de carreras de la temporada	MAX(round) por year
nombre_gran_premio	VARCHAR(100)		No	Nombre oficial del Gran Premio	racess.name
hora_inicio_utc	TIME		Sí	Hora de largada en UTC	racess.time (\N → NULL)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
tiene_sprint	BOOLEAN		No	Indica si el fin de semana tuvo carrera sprint	Existe en sprint_results
sistema_puntos	VARCHAR(60)		No	Sistema de puntuación vigente en la temporada	Regla por año (sección 5.7)
url_referencia	VARCHAR(255)		Sí	Enlace a la página de Wikipedia de la carrera	rac.es.url

Dimensión Circuito

Circuitos donde se disputaron los Grandes Premios. El continente se deriva del país en el ETL para completar la jerarquía geográfica.

Dimensión Circuito (dim_circuito)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_circuito	INTEGER	PK	No	Clave sustituta (IDENTITY); -1 = desconocido	Generada por el DBMS
id_circuito_origen	INTEGER	NK	No	Identificador del circuito en la fuente	circuits.circuitId
referencia	VARCHAR(50)		No	Código corto del circuito	circuits.circuitRef
nombre_circuito	VARCHAR(100)		No	Nombre del circuito	circuits.name
localidad	VARCHAR(100)		No	Ciudad o localidad	circuits.location
pais	VARCHAR(60)		No	País sede	circuits.country
continente	VARCHAR(20)		No	Continente del país sede	Tabla de correspondencia país → continente
latitud	NUMERIC(9,6)		Sí	Latitud geográfica	circuits.lat
longitud	NUMERIC(9,6)		Sí	Longitud geográfica	circuits.lng
altitud_m	INTEGER		Sí	Altitud sobre el nivel del mar en metros	circuits.alt (\N → NULL)

Dimensión Piloto

Pilotos que participaron en al menos un Gran Premio. Los atributos no cambian en el tiempo, por lo que se aplica la estrategia de cambios lentos de tipo 1 (una corrección sobrescribe el valor).

Dimensión Piloto (dim_piloto)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_piloto	INTEGER	PK	No	Clave sustituta (IDENTITY); -1 = desconocido	Generada por el DBMS
id_piloto_origen	INTEGER	NK	No	Identificador del piloto en la fuente	drivers.driverId
referencia	VARCHAR(50)		No	Código corto del piloto	drivers.driverRef
codigo	CHAR(3)		Sí	Código de tres letras (VER, HAM, ...)	drivers.code (\N → NULL)
numero_permanente	SMALLINT		Sí	Número permanente (desde 2014)	drivers.number (\N → NULL)
nombre	VARCHAR(60)		No	Nombre de pila	drivers.forename
apellido	VARCHAR(60)		No	Apellido	drivers.surname
nombre_completo	VARCHAR(120)		No	Nombre y apellido para reportes	forename ' ' surname
fecha_nacimiento	DATE		Sí	Fecha de nacimiento	drivers.dob

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
nacionalidad	VARCHAR(50)		No	Nacionalidad deportiva	drivers.nationality

Dimensión Escudería

Constructores o equipos. En la fuente, un cambio de nombre comercial (por ejemplo, Toro Rosso → AlphaTauri) ya figura como otro constructor, por lo que basta el tipo 1.

Dimensión Escudería (dim_escuderia)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_escuderia	INTEGER	PK	No	Clave sustituta (IDENTITY); -1 = desconocida	Generada por el DBMS
id_escuderia_origen	INTEGER	NK	No	Identificador del constructor en la fuente	constructors.constructorId
referencia	VARCHAR(50)		No	Código corto del constructor	constructors.constructorRef
nombre_escuderia	VARCHAR(100)		No	Nombre de la escudería	constructors.name
nacionalidad	VARCHAR(50)		No	Nacionalidad de la escudería	constructors.nationality

Dimensión Estado

Estado con el que el piloto terminó la carrera. La categoría agrupa los 139 estados originales en siete grupos útiles para el análisis de confiabilidad.

Dimensión Estado de finalización (dim_estado)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_estado	INTEGER	PK	No	Clave sustituta (IDENTITY); -1 = desconocido	Generada por el DBMS
id_estado_origen	INTEGER	NK	No	Identificador del estado en la fuente	status.statusId
descripcion_estado	VARCHAR(60)		No	Estado original (Finished, +1 Lap, Engine, ...)	status.status
categoria_estado	VARCHAR(40)		No	Categoría de análisis del estado	Regla de agrupación (sección 5.7)

4.3 Tabla de hechos

La tabla fact_resultado_carrera contiene las claves foráneas hacia las seis dimensiones, tres dimensiones degeneradas (atributos del evento que no justifican una tabla propia), las métricas numéricas y siete indicadores 0/1. Los indicadores se almacenan como **SMALLINT** para que puedan sumarse directamente en las consultas y en los cubos OLAP: la suma de es_victoria es el número de victorias.

Tabla de hechos Resultado de carrera (fact_resultado_carrera)

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_resultado	BIGINT	PK	No	Clave sustituta del hecho (IDENTITY)	Generada por el DBMS
sk_tiempo	INTEGER	FK	No	Fecha de la carrera	racas.date → dim_tiempo
sk_carrera	INTEGER	FK	No	Gran Premio	results.raceld → dim_carrera

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
sk_circuito	INTEGER	FK	No	Circuito de la carrera	racas.circuitId → dim_circuito
sk_piloto	INTEGER	FK	No	Piloto	results.driverId → dim_piloto
sk_escuderia	INTEGER	FK	No	Escudería con la que corrió	results.constructorId → dim_escuderia
sk_estado	INTEGER	FK	No	Estado de finalización	results.statusId → dim_estado
id_resultado_origen	INTEGER	DD	No	Identificador del resultado (único)	results.resultId
numero_auto	SMALLINT	DD	Sí	Número del auto en la carrera	results.number
codigo_posicion	VARCHAR(3)	DD	No	Posición oficial o código R, D, E, W, F, N	results.positionText
posicion_salida	SMALLINT		No	Posición de largada (0 = desde boxes)	results.grid
posicion_final	SMALLINT		Sí	Posición clasificada; nula si no fue clasificado	results.position
orden_llegada	SMALLINT		No	Orden final de todos los participantes	results.positionOrder
posiciones_ganadas	SMALLINT		Sí	Posiciones ganadas (negativo = perdidas)	grid – position (si grid > 0 y clasificado)
puntos	NUMERIC(5,2)		No	Puntos oficiales del Gran Premio	results.points
puntos_sistema_actual	NUMERIC(5,2)		No	Puntos recalculados con el sistema actual	25-18-15-12-10-8-6-4-2-1 según position
vueltas_completadas	SMALLINT		No	Vueltas completadas	results.laps
tiempo_carrera_ms	BIGINT		Sí	Tiempo total de carrera en milisegundos	results.milliseconds
tiempo_vuelta_rapida_ms	INTEGER		Sí	Mejor vuelta del piloto en milisegundos	results.fastestLapTime (m:ss.mmm → ms)
velocidad_vuelta_rapida_kmh	NUMERIC(7,3)		Sí	Velocidad media de la mejor vuelta (km/h)	results.fastestLapSpeed
ranking_vuelta_rapida	SMALLINT		Sí	Puesto de su mejor vuelta en la carrera	results.rank
edad_piloto	NUMERIC(4,1)		Sí	Edad del piloto el día de la carrera (años)	(racas.date – drivers.dob) / 365,25
puntos_acumulados_temporada	NUMERIC(6,2)		Sí	Puntos del piloto en el campeonato tras la carrera	driver_standings.points
posicion_campeonato	SMALLINT		Sí	Posición del piloto en el campeonato tras la carrera	driver_standings.position
es_victoria	SMALLINT		No	1 si ganó la carrera	posicion_final = 1
es_podio	SMALLINT		No	1 si terminó entre los tres primeros	posicion_final ≤ 3
es_pole	SMALLINT		No	1 si largó primero	posicion_salida = 1
es_en_puntos	SMALLINT		No	1 si sumó puntos	puntos > 0
es_vuelta_rapida	SMALLINT		No	1 si hizo la vuelta más rápida de la carrera	ranking_vuelta_rapida = 1
es_finalizado	SMALLINT		No	1 si fue clasificado	posicion_final no nula
es_abandono	SMALLINT		No	1 si se retiró sin ser clasificado	codigo_posicion = 'R'

Atributo	Tipo	Clave	Nulo	Descripción	Origen / regla
fecha_carga	TIMESTAMP		No	Fecha y hora de carga (auditoría del ETL)	DEFAULT CURRENT_TIMESTAMP

4.4 Claves y estrategia de dimensiones

Tipos de claves utilizados

Tipo de clave	Implementación	Ejemplo
Sustituta	Entero generado por el DBMS (<code>IDENTITY</code>) en cada dimensión; en Tiempo, clave inteligente AAAAMMDD.	sk_piloto; sk_tiempo = 20230305
Natural	Identificador de la fuente, con restricción <code>UNIQUE</code> . El ETL la usa para buscar la clave sustituta.	id_piloto_origen = drivers.driverId
Foránea	Seis columnas sk_* obligatorias en la tabla de hechos, una por dimensión.	fk_fact_resultado_piloto
Primaria del hecho	Clave sustituta sk_resultado; la unicidad del grano se garantiza con <code>UNIQUE (id_resultado_origen)</code> .	uq_fact_resultado_origen
Dimensión degenerada	Atributos del evento sin tabla propia.	id_resultado_origen, numero_auto, codigo_posicion

¿Por qué claves sustitutas? Independizan el Data Warehouse de los identificadores de la fuente, hacen más eficientes las uniones (enteros pequeños), permiten representar miembros especiales como «Desconocido» y dejan abierta la posibilidad de historizar cambios (tipo 2) en el futuro sin modificar la tabla de hechos.

Dimensiones de cambio lento: todas las dimensiones usan el **tipo 1** (sobrescritura), porque sus atributos descriptivos no cambian con el tiempo o, cuando cambian (como el nombre de una escudería), la fuente ya los registra como un miembro nuevo.

Miembro desconocido: las dimensiones Circuito, Piloto, Escudería y Estado tienen una fila con clave -1 («Desconocido»). Si un resultado referencia un valor inexistente, el ETL asigna -1 en lugar de descartar el hecho, lo que preserva los totales. Un resultado sin carrera válida se rechaza, porque la carrera determina la fecha y el circuito.

4.5 Relaciones entre tablas

Relaciones del esquema en estrella

Dimensión (1)	Columna en los hechos (N)	Restricción	Regla de negocio
dim_tiempo	sk_tiempo	fk_fact_resultado_tiempo	Cada resultado ocurre en una fecha
dim_carrera	sk_carrera	fk_fact_resultado_carrera	Cada resultado pertenece a un Gran Premio
dim_circuito	sk_circuito	fk_fact_resultado_circuito	Cada resultado se obtiene en un circuito
dim_piloto	sk_piloto	fk_fact_resultado_piloto	Cada resultado corresponde a un piloto
dim_escuderia	sk_escuderia	fk_fact_resultado_escuderia	El piloto corre con una escudería en esa carrera
dim_estado	sk_estado	fk_fact_resultado_estado	Cada resultado tiene un estado final

Todas las claves foráneas son obligatorias (**NOT NULL**) y usan la acción por defecto **NO ACTION**: no se puede eliminar un miembro de una dimensión mientras existan hechos que lo referencien.

4.6 Métricas y su clasificación

Las métricas se clasifican según las dimensiones a lo largo de las cuales pueden sumarse con significado:

Clasificación de las métricas de la tabla de hechos

Métrica	Clasificación	Agregación válida	Justificación
puntos, puntos_sistema_actual	Aditiva	SUM en todas las dimensiones	Los puntos de varias carreras, pilotos o escuderías se suman (base de los campeonatos).
vueltas_completadas	Aditiva	SUM	Total de vueltas recorridas por piloto, escudería o temporada.
posiciones_ganadas	Aditiva	SUM, AVG	Total o promedio de posiciones ganadas en una temporada o en una trayectoria.
es_victoria, es_podio, es_pole, es_en_puntos, es_vuelta_rapida, es_finalizado, es_abandono	Aditiva (indicador 0/1)	SUM = conteo	La suma del indicador cuenta victorias, podios, poles, abandonos, etc.
puntos_acumulados_temporada	Semiaditiva	SUM entre pilotos de una misma ronda; en el tiempo, último valor o MAX	Es un saldo acumulado: sumarlo entre rondas cuenta los mismos puntos varias veces.
posicion_salida, posicion_final, orden_llegada, posicion_campeonato, ranking_vuelta_rapida	No aditiva	MIN, AVG, moda, conteos	Son valores ordinales: sumar posiciones no tiene significado.
tiempo_carrera_ms	No aditiva	AVG, MIN	Depende de la longitud de cada carrera; sumar tiempos de distintos pilotos no tiene significado.
tiempo_vuelta_rapida_ms, velocidad_vuelta_rapida_kmh	No aditiva	MIN, MAX, AVG	Miden ritmo, no cantidad.
edad_piloto	No aditiva	AVG, MIN, MAX	Atributo medido en el momento del evento.
Indicadores derivados (tasas y porcentajes)	No aditivos	Se recalculan desde sus componentes	Ejemplo: tasa de abandono = $\sum \text{es_abandono} / \text{N.º participaciones}$; nunca se promedian porcentajes.

Regla para la métrica semiaditiva: si por autos compartidos un piloto tiene más de un registro en la misma carrera, puntos_acumulados_temporada y posicion_campeonato se cargan solo en el registro con mejor orden de llegada (en los demás quedan nulos), para no duplicar el saldo al sumar entre pilotos.

4.7 Verificación del modelo frente a los objetivos

La siguiente matriz comprueba que cada objetivo específico puede responderse con las métricas y atributos del modelo:

Trazabilidad entre objetivos y modelo lógico

OE	Métricas de la tabla de hechos	Dimensiones y atributos	Cubierto
OE1	es_pole, es_victoria, posiciones_ganadas	Carrera (temporada), Circuito, Escudería	✓
OE2	es_victoria, es_podio, puntos	Carrera (temporada), Tiempo (década), Escudería	✓
OE3	es_abandono, es_finalizado	Estado (categoría), Escudería, Carrera, Circuito	✓
OE4	puntos_sistema_actual, es_victoria, es_podio, edad_piloto, puntos_acumulados_temporada, posicion_campeonato	Piloto, Carrera (temporada, ronda)	✓
OE5	posiciones_ganadas, es_abandono, es_pole, es_victoria	Circuito (continente, país, circuito)	✓
OE6	tiempo_vuelta_rapida_ms, velocidad_vuelta_rapida_kmh	Circuito, Carrera (temporada)	✓

Para cada objetivo se escribió una consulta analítica (Anexo D) que se ejecutó correctamente sobre el modelo físico en PostgreSQL 16 con datos de prueba. Los resultados reales se obtendrán después de la carga ETL. Como ejemplo, la consulta del OE2 calcula la participación de cada escudería en las victorias, los podios y los puntos de cada temporada usando funciones de ventana:

```
WITH por_escuderia AS (
  SELECT c.temporada,
         e.nombre_escuderia,
         SUM(f.es_victoria) AS victorias,
         SUM(f.es_podio)   AS podios,
         SUM(f.puntos)     AS puntos
  FROM fact_resultado_carrera f
  JOIN dim_carrera   c ON c.sk_carrera = f.sk_carrera
  JOIN dim_escuderia e ON e.sk_escuderia = f.sk_escuderia
  GROUP BY c.temporada, e.nombre_escuderia
)
SELECT temporada,
       nombre_escuderia,
       victorias,
       podios,
       puntos,
       ROUND(100.0 * victorias / NULLIF(SUM(victorias) OVER (PARTITION BY temporada), 0), 1) AS pct_victorias,
       ROUND(100.0 * podios   / NULLIF(SUM(podios)   OVER (PARTITION BY temporada), 0), 1) AS pct_podios,
       ROUND(100.0 * puntos   / NULLIF(SUM(puntos)   OVER (PARTITION BY temporada), 0), 1) AS cuota_puntos
FROM por_escuderia
ORDER BY temporada, puntos DESC;
```

Y la consulta del OE4 muestra el uso correcto de la métrica semiaditiva: el campeón de cada temporada se obtiene tomando los puntos acumulados en la última ronda, sin sumarlos entre rondas:

```
SELECT DISTINCT ON (c.temporada)
       c.temporada,
       p.nombre_completo AS campeon,
       f.puntos_acumulados_temporada AS puntos_finales
FROM fact_resultado_carrera f
JOIN dim_carrera c ON c.sk_carrera = f.sk_carrera
JOIN dim_piloto p ON p.sk_piloto = f.sk_piloto
WHERE f.posicion_campeonato = 1
ORDER BY c.temporada, c.ronda DESC;
```

5. Modelado físico del DataMart

5.1 Plataforma y convenciones

Decisiones de implementación

DBMS	PostgreSQL 16: software libre, con columnas <code>IDENTITY</code> , restricciones <code>CHECK</code> , índices parciales y funciones de ventana; se conecta por JDBC con Apache Hop y con las herramientas OLAP.
Base de datos	<code>dw_formula1</code> , codificación UTF-8.
Esquema	<code>dw_f1</code> , que separa los objetos del DataMart; las tablas de preparación del ETL podrán ubicarse en otro esquema.
Nombres de tablas	Prefijo <code>dim_</code> para dimensiones y <code>fact_</code> para hechos; en español, minúsculas y sin tildes.
Nombres de columnas	<code>sk_clave</code> sustituta, <code>id*_origen</code> clave natural, <code>es_</code> indicador 0/1.
Restricciones e índices	<code>pk_</code> , <code>fk_</code> , <code>uq_</code> , <code>ck_</code> e <code>ix_</code> seguidos del nombre de la tabla y el propósito.

5.2 Tipos de datos

Criterios para la elección de tipos de datos

Dato	Tipo físico	Justificación
Claves sustitutas de dimensiones	<code>INTEGER IDENTITY</code>	Generadas por el DBMS; rango suficiente para cualquier dimensión.
Clave sustituta de hechos	<code>BIGINT IDENTITY</code>	Sin límite práctico de crecimiento.
Clave de tiempo	<code>INTEGER (AAAAMDD)</code>	Legible y ordenable; se calcula sin consultar la dimensión.
Posiciones, vueltas, rondas, indicadores	<code>SMALLINT</code>	Valores pequeños (2 bytes). Los indicadores 0/1 se pueden sumar, a diferencia de <code>BOOLEAN</code> .
Puntos	<code>NUMERIC(5,2) / (6,2)</code>	Admiten medios puntos sin errores de redondeo de punto flotante.
Tiempos	<code>BIGINT / INTEGER (ms)</code>	Permiten promedios, mínimos y diferencias, lo que no es posible con el texto «1:27.452».
Velocidad	<code>NUMERIC(7,3)</code>	Conserva la precisión de la fuente (milésimas de km/h).
Coordenadas	<code>NUMERIC(9,6)</code>	Seis decimales ($\approx 0,1$ m) dentro de los rangos válidos de latitud y longitud.
Fechas y horas	<code>DATE, TIME, TIMESTAMP</code>	Tipos nativos que permiten aritmética de fechas (por ejemplo, la edad del piloto).
Textos	<code>VARCHAR(n), CHAR(3)</code>	Longitud acotada al dominio; <code>CHAR(3)</code> para el código fijo del piloto.

5.3 Restricciones e integridad referencial

La integridad del DataMart se garantiza en el propio DBMS, de modo que ningún error del ETL pueda dejar datos inconsistentes:

- **Claves primarias** en las siete tablas y **claves naturales únicas** (`UNIQUE`) en las dimensiones; en Carrera también es única la combinación temporada y ronda.
- **Seis claves foráneas** obligatorias desde la tabla de hechos hacia cada dimensión.

- **Reglas de dominio (CHECK):** posiciones mayores que cero, puntos no negativos, edad entre 14 y 70 años, coordenadas válidas, continentes y categorías de estado de una lista cerrada, y código de posición con el patrón `^([0-9]{1,2}|R|D|E|W|F|N)$`.
- **Coherencia de los indicadores:** por ejemplo, `es_victoria = 1` si y solo si `posicion_final = 1`, y `es_abandono = 1` si y solo si `codigo_posicion = 'R'`. Así, los conteos de los cubos OLAP siempre coinciden con las posiciones.
- **Dimensión Tiempo:** la clave debe coincidir con la fecha (`sk_tiempo = AAAAMMDD`) y la década con el año.

Definición física de la tabla de hechos (el script completo está en el Anexo A):

```
CREATE TABLE fact_resultado_carrera (
  sk_resultado          BIGINT          GENERATED BY DEFAULT AS IDENTITY,
  -- Claves foráneas hacia las dimensiones
  sk_tiempo             INTEGER         NOT NULL,
  sk_carrera            INTEGER         NOT NULL,
  sk_circuito           INTEGER         NOT NULL,
  sk_piloto             INTEGER         NOT NULL,
  sk_escuderia          INTEGER         NOT NULL,
  sk_estado             INTEGER         NOT NULL,
  -- Dimensiones degeneradas
  id_resultado_origen   INTEGER         NOT NULL,
  numero_auto           SMALLINT        NULL,
  codigo_posicion       VARCHAR(3)      NOT NULL,
  -- Métricas de posición y rendimiento
  posicion_salida       SMALLINT        NOT NULL,
  posicion_final        SMALLINT        NULL,
  orden_llegada         SMALLINT        NOT NULL,
  posiciones_ganadas    SMALLINT        NULL,
  puntos                NUMERIC(5,2)   NOT NULL DEFAULT 0,
  puntos_sistema_actual NUMERIC(5,2)   NOT NULL DEFAULT 0,
  vueltas_completadas   SMALLINT        NOT NULL DEFAULT 0,
  tiempo_carrera_ms     BIGINT          NULL,
  tiempo_vuelta_rapida_ms INTEGER       NULL,
  velocidad_vuelta_rapida_kmh NUMERIC(7,3) NULL,
  ranking_vuelta_rapida SMALLINT        NULL,
  edad_piloto           NUMERIC(4,1)    NULL,
  -- Métricas del campeonato después de la carrera
  puntos_acumulados_temporada NUMERIC(6,2) NULL,
  posicion_campeonato   SMALLINT        NULL,
  -- Indicadores 0/1 (sumables en consultas y cubos OLAP)
  es_victoria           SMALLINT        NOT NULL DEFAULT 0,
  es_podio             SMALLINT        NOT NULL DEFAULT 0,
  es_pole              SMALLINT        NOT NULL DEFAULT 0,
  es_en_puntos         SMALLINT        NOT NULL DEFAULT 0,
  es_vuelta_rapida     SMALLINT        NOT NULL DEFAULT 0,
  es_finalizado        SMALLINT        NOT NULL DEFAULT 0,
  es_abandono          SMALLINT        NOT NULL DEFAULT 0,
  -- Auditoría del ETL
  fecha_carga          TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT pk_fact_resultado_carrera PRIMARY KEY (sk_resultado),
  CONSTRAINT uq_fact_resultado_origen UNIQUE (id_resultado_origen),

  -- Integridad referencial
  CONSTRAINT fk_fact_resultado_tiempo FOREIGN KEY (sk_tiempo) REFERENCES dim_tiempo (sk_tiempo),
  CONSTRAINT fk_fact_resultado_carrera FOREIGN KEY (sk_carrera) REFERENCES dim_carrera (sk_carrera),
  CONSTRAINT fk_fact_resultado_circuito FOREIGN KEY (sk_circuito) REFERENCES dim_circuito (sk_circuito),
  CONSTRAINT fk_fact_resultado_piloto FOREIGN KEY (sk_piloto) REFERENCES dim_piloto (sk_piloto),
  CONSTRAINT fk_fact_resultado_escuderia FOREIGN KEY (sk_escuderia) REFERENCES dim_escuderia (sk_escuderia),
  CONSTRAINT fk_fact_resultado_estado FOREIGN KEY (sk_estado) REFERENCES dim_estado (sk_estado),

  -- Reglas de dominio de las métricas
  CONSTRAINT ck_fact_codigo_posicion CHECK (codigo_posicion ~ '^([0-9]{1,2}|R|D|E|W|F|N)$'),
  CONSTRAINT ck_fact_posicion_salida CHECK (posicion_salida >= 0),
  CONSTRAINT ck_fact_posicion_final CHECK (posicion_final > 0),
  CONSTRAINT ck_fact_orden_llegada CHECK (orden_llegada > 0),
  CONSTRAINT ck_fact_puntos CHECK (puntos >= 0),
```

```

CONSTRAINT ck_fact_puntos_actual CHECK (puntos_sistema_actual BETWEEN 0 AND 25),
CONSTRAINT ck_fact_vueltas CHECK (vueltas_completadas >= 0),
CONSTRAINT ck_fact_tiempo_carrera CHECK (tiempo_carrera_ms > 0),
CONSTRAINT ck_fact_tiempo_vuelta CHECK (tiempo_vuelta_rapida_ms > 0),
CONSTRAINT ck_fact_velocidad CHECK (velocidad_vuelta_rapida_kmh > 0),
CONSTRAINT ck_fact_ranking_vuelta CHECK (ranking_vuelta_rapida > 0),
CONSTRAINT ck_fact_edad CHECK (edad_piloto BETWEEN 14 AND 70),
CONSTRAINT ck_fact_puntos_acumulados CHECK (puntos_acumulados_temporada >= 0),
CONSTRAINT ck_fact_posicion_campeonato CHECK (posicion_campeonato > 0),

-- Indicadores binarios y coherencia con las métricas de origen
CONSTRAINT ck_fact_indicadores CHECK (
    es_victoria IN (0, 1) AND es_podio IN (0, 1) AND es_pole IN (0, 1) AND
    es_en_puntos IN (0, 1) AND es_vuelta_rapida IN (0, 1) AND
    es_finalizado IN (0, 1) AND es_abandono IN (0, 1)),
CONSTRAINT ck_fact_victoria CHECK ((es_victoria = 1) = (COALESCE(posicion_final, 0) = 1)),
CONSTRAINT ck_fact_podio CHECK ((es_podio = 1) = (COALESCE(posicion_final, 99) <= 3)),
CONSTRAINT ck_fact_pole CHECK ((es_pole = 1) = (posicion_salida = 1)),
CONSTRAINT ck_fact_en_puntos CHECK ((es_en_puntos = 1) = (puntos > 0)),
CONSTRAINT ck_fact_vuelta_rapida CHECK ((es_vuelta_rapida = 1) = (COALESCE(ranking_vuelta_rapida, 0) = 1)),
CONSTRAINT ck_fact_finalizado CHECK ((es_finalizado = 1) = (posicion_final IS NOT NULL)),
CONSTRAINT ck_fact_abandono CHECK ((es_abandono = 1) = (codigo_posicion = 'R'))
);

```

5.4 Índices

Además de los índices que PostgreSQL crea automáticamente para las claves primarias y las restricciones UNIQUE (usados por el ETL para buscar claves sustitutas), se definieron quince índices orientados a las consultas analíticas:

Índices del DataMart

Índice	Tabla (columnas)	Propósito
ix_fact_resultado_tiempo, _carrera, _circuito, _estado	fact (sk_tiempo), (sk_carrera), (sk_circuito), (sk_estado)	Uniones y filtros por dimensión
ix_fact_resultado_piloto_tiempo	fact (sk_piloto, sk_tiempo)	Series históricas por piloto; también cubre los filtros por piloto
ix_fact_resultado_escuderia_tiempo	fact (sk_escuderia, sk_tiempo)	Series históricas por escudería
ix_fact_resultado_victorias	fact (sk_escuderia, sk_piloto) WHERE es_victoria = 1	Índice parcial para rankings de victorias (≈ 4 % de las filas)
ix_dim_tiempo_anio_mes, ix_dim_tiempo_decada	dim_tiempo (anio, mes), (decada)	Navegación de la jerarquía temporal
ix_dim_carrera_temporada	dim_carrera (temporada)	Filtros por temporada
ix_dim_circuito_pais	dim_circuito (continente, pais)	Jerarquía geográfica
ix_dim_piloto_nacionalidad, ix_dim_piloto_apellido	dim_piloto (nacionalidad), (apellido)	Filtros y búsqueda de pilotos
ix_dim_escuderia_nombre	dim_escuderia (nombre_escuderia)	Filtros por escudería
ix_dim_estado_categoria	dim_estado (categoria_estado)	Agrupación de causas de abandono

5.5 Scripts de implementación

Scripts SQL entregados (carpeta sql/)

Orden	Script	Contenido
1	01_crear_esquema_y_tablas.sql	Esquema dw_f1, seis dimensiones y tabla de hechos con PK, FK, UNIQUE, CHECK y comentarios (Anexo A)
2	02_crear_indices.sql	Quince índices analíticos (Anexo B)
3	03_datos_iniciales.sql	Miembros desconocidos (-1) y los 29 585 días de la dimensión Tiempo (Anexo C)
4	04_consultas_objetivos.sql	Consultas analíticas de verificación de los objetivos (Anexo D)
5	05_pruebas_integridad.sql	Pruebas de integridad; deshace los cambios con ROLLBACK (Anexo E)

```
createdb -U postgres -E UTF8 -T template0 dw_formulal
psql -U postgres -d dw_formulal -f 01_crear_esquema_y_tablas.sql
psql -U postgres -d dw_formulal -f 02_crear_indices.sql
psql -U postgres -d dw_formulal -f 03_datos_iniciales.sql
psql -U postgres -d dw_formulal -f 05_pruebas_integridad.sql
```

5.6 Evidencia de la implementación

Los scripts se ejecutaron en PostgreSQL 16.15. La siguiente salida, obtenida de los catálogos del sistema, confirma las tablas creadas con sus claves, restricciones e índices, las seis claves foráneas y las filas generadas por el script de datos iniciales:

```
-- Servidor: PostgreSQL 16.15 · base de datos dw_formulal · esquema dw_f1
dw_formulal=# -- 1. Tablas creadas en el esquema dw_f1
          tabla          | columnas | pk | fk | unique | check | indices
-----+-----+---+---+-----+-----+-----
dim_carrera              |         | 10 | 1 | 0       | 2      | 4
dim_circuito             |         | 10 | 1 | 0       | 1      | 3
dim_escuderia            |         | 5  | 1 | 0       | 1      | 3
dim_estado               |         | 4  | 1 | 0       | 1      | 3
dim_piloto               |         | 10 | 1 | 0       | 1      | 4
dim_tiempo               |         | 12 | 1 | 0       | 1      | 7
fact_resultado_carrera   |         | 32 | 1 | 6       | 1      | 22
fact_resultado_carrera   |         | 32 | 1 | 6       | 1      | 22

dw_formulal=# -- 2. Claves foráneas (integridad referencial hechos -> dimensiones)
          restriccion    | tabla          | referencia
-----+-----+-----
fk_fact_resultado_carrera | dw_f1.fact_resultado_carrera | dw_f1.dim_carrera
fk_fact_resultado_circuito | dw_f1.fact_resultado_carrera | dw_f1.dim_circuito
fk_fact_resultado_escuderia | dw_f1.fact_resultado_carrera | dw_f1.dim_escuderia
fk_fact_resultado_estado   | dw_f1.fact_resultado_carrera | dw_f1.dim_estado
fk_fact_resultado_piloto   | dw_f1.fact_resultado_carrera | dw_f1.dim_piloto
fk_fact_resultado_tiempo   | dw_f1.fact_resultado_carrera | dw_f1.dim_tiempo

dw_formulal=# -- 3. Filas cargadas por el script 03 (miembros desconocidos y calendario)
          tabla          | filas | desde      | hasta
-----+-----+-----+-----
dim_tiempo              | 29585 | 1950-01-01 | 2030-12-31
dim_circuito             | 1     |             |
dim_piloto              | 1     |             |
dim_escuderia           | 1     |             |
dim_estado              | 1     |             |
```

Para comprobar la integridad referencial y las reglas de dominio se ejecutó el script de pruebas. Cada operación inválida es rechazada por el DBMS con la restricción correspondiente, y al final solo permanece el registro válido de ejemplo, que se descarta con `ROLLBACK`:

```
$ psql -U postgres -d dw_formula1 -f 05_pruebas_integridad.sql
SET
BEGIN
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
--- Prueba 1: clave foránea inexistente (sk_piloto = 9999) -> debe fallar
psql:05_pruebas_integridad.sql:37: ERROR: insert or update on table "fact_resultado_carrera" violates
foreign key constraint "fk_fact_resultado_piloto"
DETAIL: Key (sk_piloto)=(9999) is not present in table "dim_piloto".
--- Prueba 2: indicador incoherente (es_victoria = 1 con posicion_final = 2) -> debe fallar
psql:05_pruebas_integridad.sql:43: ERROR: new row for relation "fact_resultado_carrera" violates check
constraint "ck_fact_victoria"
DETAIL: Failing row contains (...).
--- Prueba 3: resultado duplicado (id_resultado_origen repetido) -> debe fallar
psql:05_pruebas_integridad.sql:48: ERROR: duplicate key value violates unique constraint
"uq_fact_resultado_origen"
DETAIL: Key (id_resultado_origen)=(99001) already exists.
--- Prueba 4: borrar un piloto que tiene hechos asociados -> debe fallar
psql:05_pruebas_integridad.sql:51: ERROR: update or delete on table "dim_piloto" violates foreign key
constraint "fk_fact_resultado_piloto" on table "fact_resultado_carrera"
DETAIL: Key (sk_piloto)=(1000) is still referenced from table "fact_resultado_carrera".
--- Prueba 5: puntos negativos -> debe fallar
psql:05_pruebas_integridad.sql:56: ERROR: new row for relation "fact_resultado_carrera" violates check
constraint "ck_fact_puntos"
DETAIL: Failing row contains (...).
--- Control: la tabla de hechos conserva solo el registro válido de ejemplo
 id_resultado_origen | codigo_posicion | puntos | es_victoria
-----+-----+-----+-----
          99001 | 1                | 25.00 | 1
(1 row)

ROLLBACK
```

5.7 Preparación para la carga ETL

La estructura física está pensada para la carga con Apache Hop en la segunda presentación:

1. **Orden de carga:** (1) `dim_tiempo`, ya generada por SQL; (2) dimensiones Estado, Circuito, Piloto, Escudería y Carrera; (3) tabla de hechos.
2. **Búsqueda de claves:** cada fila de `results.csv` obtiene sus claves sustitutas buscando la clave natural (`id*_origen`) en cada dimensión; si no existe, se asigna -1.
3. **Recarga sin duplicados:** `UNIQUE (id_resultado_origen)` impide duplicar resultados si el proceso se ejecuta más de una vez; el ETL usará inserción o actualización según la clave natural.
4. **Transformaciones previstas:** `\N` → `NULL`; tiempos «m:ss.mmm» → milisegundos; cálculo de posiciones ganadas, indicadores, puntos normalizados y edad; continente según el país; categoría del estado; sistema de puntos según la temporada; puntos acumulados desde `driver_standings` por (`raceld`, `driverId`).

Regla para dim_carrera.sistema_puntos (puntos del 1.º al 10.º)

Temporadas	Puntos por posición	Vuelta más rápida
1950–1959	8-6-4-3-2	+1 punto
1960	8-6-4-3-2-1	—
1961–1990	9-6-4-3-2-1	—
1991–2002	10-6-4-3-2-1	—
2003–2009	10-8-6-5-4-3-2-1	—
2010–2018	25-18-15-12-10-8-6-4-2-1	—
2019–2024	25-18-15-12-10-8-6-4-2-1	+1 punto si termina entre los 10 primeros
2025 en adelante	25-18-15-12-10-8-6-4-2-1	—

El sistema 2010–actual es la base de puntos_sistema_actual. Hasta 1990 solo contaban los mejores resultados de cada piloto (descartes); por eso los puntos acumulados del campeonato se toman de la fuente y no se recalculan.

Regla para dim_estado.categoria_estado

Categoría	Estados de origen (ejemplos)
Finalizó	Finished
Finalizó con vueltas de retraso	+1 Lap, +2 Laps, ...
Falla mecánica	Engine, Gearbox, Transmission, Hydraulics, Brakes, Suspension, Electrical, Power Unit, ...
Accidente o colisión	Accident, Collision, Collision damage, Spun off, ...
Descalificación	Disqualified, Excluded
No largó o no clasificó	Did not qualify, Did not prequalify, Withdrew, Not classified, 107% Rule
Otro	Estados restantes (por ejemplo, Retired sin causa especificada o enfermedad del piloto)

Conclusiones y próximos pasos

- Se seleccionó un tema con un problema analítico propio —el rendimiento competitivo en la Fórmula 1— y un conjunto de datos público con setenta y cinco temporadas, varios procesos de negocio y problemas reales de calidad de datos.
- Se definieron un objetivo general y seis objetivos específicos medibles, cada uno con indicadores y fórmulas basados en las métricas del modelo.
- De tres DataMarts propuestos se seleccionó el de Rendimiento en Carrera mediante una evaluación ponderada; su grano atómico es el resultado de un piloto con un auto en un Gran Premio.
- El modelo lógico en estrella cuenta con seis dimensiones conformadas, claves sustitutas, métricas aditivas, semiaditivas y no aditivas, y una matriz de trazabilidad que confirma que cubre los seis objetivos.
- El modelo físico está implementado y probado en PostgreSQL 16, con integridad referencial, reglas de dominio, índices analíticos y la dimensión Tiempo ya generada.

Próximos pasos (segunda presentación): desarrollar los procesos ETL con Apache Hop para extraer los CSV, depurarlos y cargar el DataMart; crear el cubo OLAP con las jerarquías definidas; construir al menos tres reportes que respondan a los objetivos; y diseñar los cuadros de mando interactivos con los KPI de la sección 2.3.

Referencias

Ergast Developer API. (s. f.). *Ergast Motor Racing Developer API: documentación y base de datos histórica de la Fórmula 1*. <http://ergast.com/mrd/>

Kimball, R. y Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3.ª ed.). Wiley.

Rao, R. (2024). *Formula 1 World Championship (1950 – 2024)* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020>

The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>

Anexos: scripts SQL

Anexo A · 01_crear_esquema_y_tablas.sql

```
-- =====
-- Proyecto Base de Datos 3 - Data Warehouse de Fórmula 1
-- DataMart seleccionado: Rendimiento en Carrera
-- Script 01: esquema, tablas de dimensiones y tabla de hechos
-- DBMS: PostgreSQL 16
--
-- Ejecución:
-- createdb -U postgres -E UTF8 -T template0 dw_formula1
-- psql -U postgres -d dw_formula1 -f 01_crear_esquema_y_tablas.sql
--
-- ATENCIÓN: el script elimina y vuelve a crear el esquema dw_f1.
-- =====

DROP SCHEMA IF EXISTS dw_f1 CASCADE;
CREATE SCHEMA dw_f1;
COMMENT ON SCHEMA dw_f1 IS 'DataMart Rendimiento en Carrera - Campeonato Mundial de Fórmula 1 (1950 en adelante)';

SET search_path TO dw_f1;

-- -----
-- DIMENSIÓN TIEMPO
-- Clave inteligente AAAAMDD; se genera completa antes del ETL.
-- Jerarquía: Década > Año > Semestre > Trimestre > Mes > Fecha
-- -----
CREATE TABLE dim_tiempo (
    sk_tiempo          INTEGER          NOT NULL,
    fecha              DATE             NOT NULL,
    dia                SMALLINT         NOT NULL,
    dia_semana         SMALLINT         NOT NULL,
    nombre_dia         VARCHAR(10)      NOT NULL,
    mes                SMALLINT         NOT NULL,
    nombre_mes         VARCHAR(10)      NOT NULL,
    trimestre          SMALLINT         NOT NULL,
    semestre           SMALLINT         NOT NULL,
    anio               SMALLINT         NOT NULL,
    decada             SMALLINT         NOT NULL,
    nombre_decada      VARCHAR(15)      NOT NULL,
    CONSTRAINT pk_dim_tiempo PRIMARY KEY (sk_tiempo),
    CONSTRAINT uq_dim_tiempo_fecha UNIQUE (fecha),
    CONSTRAINT ck_dim_tiempo_sk CHECK (sk_tiempo = EXTRACT(YEAR FROM fecha) * 10000
                                        + EXTRACT(MONTH FROM fecha) * 100
                                        + EXTRACT(DAY FROM fecha)),
    CONSTRAINT ck_dim_tiempo_dia CHECK (dia BETWEEN 1 AND 31),
    CONSTRAINT ck_dim_tiempo_dia_semana CHECK (dia_semana BETWEEN 1 AND 7),
    CONSTRAINT ck_dim_tiempo_mes CHECK (mes BETWEEN 1 AND 12),
    CONSTRAINT ck_dim_tiempo_trimestre CHECK (trimestre BETWEEN 1 AND 4),
    CONSTRAINT ck_dim_tiempo_semestre CHECK (semestre IN (1, 2)),
    CONSTRAINT ck_dim_tiempo_decada CHECK (decada = anio - anio % 10)
);

COMMENT ON TABLE dim_tiempo IS 'Dimensión calendario: un registro por día';
COMMENT ON COLUMN dim_tiempo.sk_tiempo IS 'Clave sustituta inteligente con formato AAAAMDD';
COMMENT ON COLUMN dim_tiempo.dia_semana IS 'Día de la semana ISO: 1 = lunes ... 7 = domingo';
COMMENT ON COLUMN dim_tiempo.decada IS 'Primer año de la década (1950, 1960, ...)';

-- -----
-- DIMENSIÓN CARRERA (Gran Premio)
-- Fuente: races.csv, sprint_results.csv
-- Jerarquía: Temporada > Ronda (Gran Premio)
-- -----
CREATE TABLE dim_carrera (
    sk_carrera          INTEGER          GENERATED BY DEFAULT AS IDENTITY,
    id_carrera_origen   INTEGER          NOT NULL,
    temporada           SMALLINT         NOT NULL,
    ronda               SMALLINT         NOT NULL,
    total_rondas_temporada SMALLINT      NOT NULL,
```

```

    nombre_gran_premio    VARCHAR(100) NOT NULL,
    hora_inicio_utc       TIME          NULL,
    tiene_sprint           BOOLEAN       NOT NULL DEFAULT FALSE,
    sistema_puntos         VARCHAR(60)   NOT NULL,
    url_referencia         VARCHAR(255)   NULL,
    CONSTRAINT pk_dim_carrera PRIMARY KEY (sk_carrera),
    CONSTRAINT uq_dim_carrera_origen UNIQUE (id_carrera_origen),
    CONSTRAINT uq_dim_carrera_temporada_ronda UNIQUE (temporada, ronda),
    CONSTRAINT ck_dim_carrera_temporada CHECK (temporada >= 1950),
    CONSTRAINT ck_dim_carrera_ronda CHECK (ronda BETWEEN 1 AND total_rondas_temporada)
);

COMMENT ON TABLE dim_carrera IS 'Dimensión Gran Premio: un registro por carrera del campeonato';
COMMENT ON COLUMN dim_carrera.id_carrera_origen IS 'Clave natural: races.raceId';
COMMENT ON COLUMN dim_carrera.total_rondas_temporada IS 'Cantidad de carreras de la temporada (derivado en el ETL)';
COMMENT ON COLUMN dim_carrera.tiene_sprint IS 'Verdadero si el fin de semana incluyó carrera sprint';
COMMENT ON COLUMN dim_carrera.sistema_puntos IS 'Sistema de puntuación vigente en la temporada (derivado en el ETL)';

-----
-- DIMENSIÓN CIRCUITO
-- Fuente: circuits.csv (+ tabla de correspondencia país -> continente)
-- Jerarquía: Continente > País > Localidad > Circuito
-----

CREATE TABLE dim_circuito (
    sk_circuito            INTEGER        GENERATED BY DEFAULT AS IDENTITY,
    id_circuito_origen     INTEGER        NOT NULL,
    referencia             VARCHAR(50)    NOT NULL,
    nombre_circuito        VARCHAR(100)   NOT NULL,
    localidad              VARCHAR(100)   NOT NULL,
    pais                   VARCHAR(60)    NOT NULL,
    continente             VARCHAR(20)    NOT NULL,
    latitud                NUMERIC(9,6)   NULL,
    longitud               NUMERIC(9,6)   NULL,
    altitud_m              INTEGER        NULL,
    CONSTRAINT pk_dim_circuito PRIMARY KEY (sk_circuito),
    CONSTRAINT uq_dim_circuito_origen UNIQUE (id_circuito_origen),
    CONSTRAINT ck_dim_circuito_continente CHECK (continente IN
        ('África', 'América del Norte', 'América del Sur', 'Asia', 'Europa', 'Oceanía', 'Desconocido')),
    CONSTRAINT ck_dim_circuito_latitud CHECK (latitud BETWEEN -90 AND 90),
    CONSTRAINT ck_dim_circuito_longitud CHECK (longitud BETWEEN -180 AND 180)
);

COMMENT ON TABLE dim_circuito IS 'Dimensión circuito con su ubicación geográfica';
COMMENT ON COLUMN dim_circuito.id_circuito_origen IS 'Clave natural: circuits.circuitId';
COMMENT ON COLUMN dim_circuito.continente IS 'Derivado en el ETL a partir del país';

-----
-- DIMENSIÓN PILOTO
-- Fuente: drivers.csv
-- Jerarquía: Nacionalidad > Piloto
-----

CREATE TABLE dim_piloto (
    sk_piloto              INTEGER        GENERATED BY DEFAULT AS IDENTITY,
    id_piloto_origen       INTEGER        NOT NULL,
    referencia             VARCHAR(50)    NOT NULL,
    codigo                 CHAR(3)        NULL,
    numero_permanente      SMALLINT       NULL,
    nombre                 VARCHAR(60)    NOT NULL,
    apellido               VARCHAR(60)    NOT NULL,
    nombre_completo        VARCHAR(120)   NOT NULL,
    fecha_nacimiento       DATE           NULL,
    nacionalidad           VARCHAR(50)    NOT NULL,
    CONSTRAINT pk_dim_piloto PRIMARY KEY (sk_piloto),
    CONSTRAINT uq_dim_piloto_origen UNIQUE (id_piloto_origen),
    CONSTRAINT ck_dim_piloto_numero CHECK (numero_permanente BETWEEN 0 AND 99)
);

COMMENT ON TABLE dim_piloto IS 'Dimensión piloto (SCD tipo 1)';
COMMENT ON COLUMN dim_piloto.id_piloto_origen IS 'Clave natural: drivers.driverId';
COMMENT ON COLUMN dim_piloto.codigo IS 'Código de tres letras (ej. VER, HAM); nulo en pilotos antiguos';
COMMENT ON COLUMN dim_piloto.numero_permanente IS 'Número permanente (desde 2014); nulo en pilotos antiguos';

-----
-- DIMENSIÓN ESCUDERÍA (constructor)

```

```
-- Fuente: constructors.csv
-- Jerarquía: Nacionalidad > Escudería
-----

CREATE TABLE dim_escuderia (
  sk_escuderia          INTEGER          GENERATED BY DEFAULT AS IDENTITY,
  id_escuderia_origen   INTEGER          NOT NULL,
  referencia            VARCHAR(50)      NOT NULL,
  nombre_escuderia      VARCHAR(100)    NOT NULL,
  nacionalidad          VARCHAR(50)      NOT NULL,
  CONSTRAINT pk_dim_escuderia PRIMARY KEY (sk_escuderia),
  CONSTRAINT uq_dim_escuderia_origen UNIQUE (id_escuderia_origen)
);

COMMENT ON TABLE dim_escuderia IS 'Dimensión escudería o constructor (SCD tipo 1)';
COMMENT ON COLUMN dim_escuderia.id_escuderia_origen IS 'Clave natural: constructors.constructorId';

-----

-- DIMENSIÓN ESTADO DE FINALIZACIÓN
-- Fuente: status.csv (+ regla de clasificación por categoría)
-- Jerarquía: Categoría > Estado
-----

CREATE TABLE dim_estado (
  sk_estado            INTEGER          GENERATED BY DEFAULT AS IDENTITY,
  id_estado_origen     INTEGER          NOT NULL,
  descripcion_estado   VARCHAR(60)     NOT NULL,
  categoria_estado     VARCHAR(40)     NOT NULL,
  CONSTRAINT pk_dim_estado PRIMARY KEY (sk_estado),
  CONSTRAINT uq_dim_estado_origen UNIQUE (id_estado_origen),
  CONSTRAINT ck_dim_estado_categoria CHECK (categoria_estado IN
    ('Finalizó', 'Finalizó con vueltas de retraso', 'Falla mecánica',
     'Accidente o colisión', 'Descalificación', 'No largó o no clasificó',
     'Otro', 'Desconocido'))
);

COMMENT ON TABLE dim_estado IS 'Dimensión estado de finalización del piloto en la carrera';
COMMENT ON COLUMN dim_estado.id_estado_origen IS 'Clave natural: status.statusId';
COMMENT ON COLUMN dim_estado.categoria_estado IS 'Agrupación del estado original (derivada en el ETL)';

-----

-- TABLA DE HECHOS: RESULTADO DE CARRERA
-- Fuente: results.csv (+ races, drivers, driver_standings)
-- Granularidad: un registro por cada resultado de un piloto con un auto
-- en un Gran Premio del campeonato (results.resultId).
-----

CREATE TABLE fact_resultado_carrera (
  sk_resultado          BIGINT          GENERATED BY DEFAULT AS IDENTITY,
  -- Claves foráneas hacia las dimensiones
  sk_tiempo             INTEGER          NOT NULL,
  sk_carrera            INTEGER          NOT NULL,
  sk_circuito           INTEGER          NOT NULL,
  sk_piloto             INTEGER          NOT NULL,
  sk_escuderia          INTEGER          NOT NULL,
  sk_estado             INTEGER          NOT NULL,
  -- Dimensiones degeneradas
  id_resultado_origen   INTEGER          NOT NULL,
  numero_auto           SMALLINT         NULL,
  codigo_posicion       VARCHAR(3)      NOT NULL,
  -- Métricas de posición y rendimiento
  posicion_salida       SMALLINT         NOT NULL,
  posicion_final        SMALLINT         NULL,
  orden_llegada         SMALLINT         NOT NULL,
  posiciones_ganadas    SMALLINT         NULL,
  puntos               NUMERIC(5,2)     NOT NULL DEFAULT 0,
  puntos_sistema_actual NUMERIC(5,2)     NOT NULL DEFAULT 0,
  vueltas_completadas   SMALLINT         NOT NULL DEFAULT 0,
  tiempo_carrera_ms     BIGINT           NULL,
  tiempo_vuelta_rapida_ms INTEGER        NULL,
  velocidad_vuelta_rapida_kmh NUMERIC(7,3) NULL,
  ranking_vuelta_rapida SMALLINT         NULL,
  edad_piloto           NUMERIC(4,1)     NULL,
  -- Métricas del campeonato después de la carrera
  puntos_acumulados_temporada NUMERIC(6,2) NULL,
  posicion_campeonato   SMALLINT         NULL,
  -- Indicadores 0/1 (sumables en consultas y cubos OLAP)
```

```

    es_victoria          SMALLINT      NOT NULL DEFAULT 0,
    es_podio             SMALLINT      NOT NULL DEFAULT 0,
    es_pole              SMALLINT      NOT NULL DEFAULT 0,
    es_en_puntos         SMALLINT      NOT NULL DEFAULT 0,
    es_vuelta_rapida     SMALLINT      NOT NULL DEFAULT 0,
    es_finalizado        SMALLINT      NOT NULL DEFAULT 0,
    es_abandono          SMALLINT      NOT NULL DEFAULT 0,
    -- Auditoría del ETL
    fecha_carga          TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_fact_resultado_carrera PRIMARY KEY (sk_resultado),
    CONSTRAINT uq_fact_resultado_origen UNIQUE (id_resultado_origen),

    -- Integridad referencial
    CONSTRAINT fk_fact_resultado_tiempo FOREIGN KEY (sk_tiempo) REFERENCES dim_tiempo (sk_tiempo),
    CONSTRAINT fk_fact_resultado_carrera FOREIGN KEY (sk_carrera) REFERENCES dim_carrera (sk_carrera),
    CONSTRAINT fk_fact_resultado_circuito FOREIGN KEY (sk_circuito) REFERENCES dim_circuito (sk_circuito),
    CONSTRAINT fk_fact_resultado_piloto FOREIGN KEY (sk_piloto) REFERENCES dim_piloto (sk_piloto),
    CONSTRAINT fk_fact_resultado_escuderia FOREIGN KEY (sk_escuderia) REFERENCES dim_escuderia (sk_escuderia),
    CONSTRAINT fk_fact_resultado_estado FOREIGN KEY (sk_estado) REFERENCES dim_estado (sk_estado),

    -- Reglas de dominio de las métricas
    CONSTRAINT ck_fact_codigo_posicion CHECK (codigo_posicion ~ '^[0-9]{1,2}|R|D|E|W|F|N$'),
    CONSTRAINT ck_fact_posicion_salida CHECK (posicion_salida >= 0),
    CONSTRAINT ck_fact_posicion_final CHECK (posicion_final > 0),
    CONSTRAINT ck_fact_orden_llegada CHECK (orden_llegada > 0),
    CONSTRAINT ck_fact_puntos CHECK (puntos >= 0),
    CONSTRAINT ck_fact_puntos_actual CHECK (puntos_sistema_actual BETWEEN 0 AND 25),
    CONSTRAINT ck_fact_vueltas CHECK (vueltas_completadas >= 0),
    CONSTRAINT ck_fact_tiempo_carrera CHECK (tiempo_carrera_ms > 0),
    CONSTRAINT ck_fact_tiempo_vuelta CHECK (tiempo_vuelta_rapida_ms > 0),
    CONSTRAINT ck_fact_velocidad CHECK (velocidad_vuelta_rapida_kmh > 0),
    CONSTRAINT ck_fact_ranking_vuelta CHECK (ranking_vuelta_rapida > 0),
    CONSTRAINT ck_fact_edad CHECK (edad_piloto BETWEEN 14 AND 70),
    CONSTRAINT ck_fact_puntos_acumulados CHECK (puntos_acumulados_temporada >= 0),
    CONSTRAINT ck_fact_posicion_campeonato CHECK (posicion_campeonato > 0),

    -- Indicadores binarios y coherencia con las métricas de origen
    CONSTRAINT ck_fact_indicadores CHECK (
        es_victoria IN (0, 1) AND es_podio IN (0, 1) AND es_pole IN (0, 1) AND
        es_en_puntos IN (0, 1) AND es_vuelta_rapida IN (0, 1) AND
        es_finalizado IN (0, 1) AND es_abandono IN (0, 1)),
    CONSTRAINT ck_fact_victoria CHECK ((es_victoria = 1) = (COALESCE(posicion_final, 0) = 1)),
    CONSTRAINT ck_fact_podio CHECK ((es_podio = 1) = (COALESCE(posicion_final, 99) <= 3)),
    CONSTRAINT ck_fact_pole CHECK ((es_pole = 1) = (posicion_salida = 1)),
    CONSTRAINT ck_fact_en_puntos CHECK ((es_en_puntos = 1) = (puntos > 0)),
    CONSTRAINT ck_fact_vuelta_rapida CHECK ((es_vuelta_rapida = 1) = (COALESCE(ranking_vuelta_rapida, 0) = 1)),
    CONSTRAINT ck_fact_finalizado CHECK ((es_finalizado = 1) = (posicion_final IS NOT NULL)),
    CONSTRAINT ck_fact_abandono CHECK ((es_abandono = 1) = (codigo_posicion = 'R'))
);

COMMENT ON TABLE fact_resultado_carrera IS 'Hechos: resultado de un piloto con un auto en un Gran Premio';
COMMENT ON COLUMN fact_resultado_carrera.id_resultado_origen IS 'Dimensión degenerada: results.resultId';
COMMENT ON COLUMN fact_resultado_carrera.numero_auto IS 'Dimensión degenerada: número del auto en la carrera';
COMMENT ON COLUMN fact_resultado_carrera.codigo_posicion IS 'Dimensión degenerada: posición oficial o código R/D/E/W/F/N';
COMMENT ON COLUMN fact_resultado_carrera.posicion_salida IS 'Posición en la grilla de salida (0 = largó desde boxes o sin dato). No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.posicion_final IS 'Posición final clasificada; nula si no fue clasificado. No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.orden_llegada IS 'Orden final de todos los participantes, incluidos no clasificados. No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.posiciones_ganadas IS 'posicion_salida - posicion_final para clasificados que largaron desde la grilla. Aditiva';
COMMENT ON COLUMN fact_resultado_carrera.puntos IS 'Puntos oficiales obtenidos en el Gran Premio. Aditiva';
COMMENT ON COLUMN fact_resultado_carrera.puntos_sistema_actual IS 'Puntos recalculados con el sistema 25-18-15-12-10-8-6-4-2-1. Aditiva';
COMMENT ON COLUMN fact_resultado_carrera.vueltas_completadas IS 'Vueltas completadas en la carrera. Aditiva';
COMMENT ON COLUMN fact_resultado_carrera.tiempo_carrera_ms IS 'Tiempo total de carrera en milisegundos (solo en la vuelta del líder). No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.tiempo_vuelta_rapida_ms IS 'Mejor vuelta del piloto en milisegundos (desde 2004). No aditiva';

```

```

COMMENT ON COLUMN fact_resultado_carrera.velocidad_vuelta_rapida_kmh IS 'Velocidad media de la mejor vuelta en km/h (desde 2004). No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.ranking_vuelta_rapida IS 'Ranking de la mejor vuelta dentro de la carrera (1 = más rápida). No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.edad_piloto IS 'Edad del piloto en años a la fecha de la carrera. No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.puntos_acumulados_temporada IS 'Puntos del piloto en el campeonato tras la carrera (incluye sprints). Semiaditiva';
COMMENT ON COLUMN fact_resultado_carrera.posicion_campeonato IS 'Posición del piloto en el campeonato tras la carrera. No aditiva';
COMMENT ON COLUMN fact_resultado_carrera.es_pole IS '1 si largó desde la primera posición de la grilla';
COMMENT ON COLUMN fact_resultado_carrera.es_abandono IS '1 si se retiró sin ser clasificado (codigo_posicion = R)';

```

Anexo B · 02_crear_indices.sql

```

-- =====
-- Proyecto Base de Datos 3 - Data Warehouse de Fórmula 1
-- Script 02: índices orientados a consultas analíticas
--
-- Las claves primarias y las restricciones UNIQUE sobre las claves
-- naturales (id*_origen) ya crean sus propios índices; estos últimos
-- son los que usa el ETL para buscar la clave sustituta de cada fila.
-- =====

SET search_path TO dw_f1;

-- Tabla de hechos: índice por clave foránea (filtros y uniones)
CREATE INDEX ix_fact_resultado_tiempo ON fact_resultado_carrera (sk_tiempo);
CREATE INDEX ix_fact_resultado_carrera ON fact_resultado_carrera (sk_carrera);
CREATE INDEX ix_fact_resultado_circuito ON fact_resultado_carrera (sk_circuito);
CREATE INDEX ix_fact_resultado_estado ON fact_resultado_carrera (sk_estado);

-- Series de tiempo por piloto y por escudería (evolución por temporada).
-- Como sk_piloto y sk_escuderia son la primera columna, estos índices
-- también cubren los filtros por piloto o por escudería.
CREATE INDEX ix_fact_resultado_piloto_tiempo ON fact_resultado_carrera (sk_piloto, sk_tiempo);
CREATE INDEX ix_fact_resultado_escuderia_tiempo ON fact_resultado_carrera (sk_escuderia, sk_tiempo);

-- Índice parcial para el análisis de victorias (pocas filas, consultas frecuentes)
CREATE INDEX ix_fact_resultado_victorias ON fact_resultado_carrera (sk_escuderia, sk_piloto)
WHERE es_victoria = 1;

-- Dimensiones: atributos usados como niveles de jerarquía o filtros
CREATE INDEX ix_dim_tiempo_anio_mes ON dim_tiempo (anio, mes);
CREATE INDEX ix_dim_tiempo_decada ON dim_tiempo (decada);
CREATE INDEX ix_dim_carrera_temporada ON dim_carrera (temporada);
CREATE INDEX ix_dim_circuito_pais ON dim_circuito (continente, pais);
CREATE INDEX ix_dim_piloto_nacionalidad ON dim_piloto (nacionalidad);
CREATE INDEX ix_dim_piloto_apellido ON dim_piloto (apellido);
CREATE INDEX ix_dim_escuderia_nombre ON dim_escuderia (nombre_escuderia);
CREATE INDEX ix_dim_estado_categoria ON dim_estado (categoria_estado);

```

Anexo C · 03_datos_iniciales.sql

```

-- =====
-- Proyecto Base de Datos 3 - Data Warehouse de Fórmula 1
-- Script 03: datos iniciales previos al ETL
-- 1. Miembros "Desconocido" (clave -1) para búsquedas sin coincidencia
-- 2. Generación completa de la dimensión tiempo (1950-2030)
-- =====

SET search_path TO dw_f1;

-- =====

-- 1. Miembros desconocidos
-- El ETL asigna la clave -1 cuando un resultado referencia un circuito,
-- piloto, escudería o estado que no existe en la dimensión, en lugar de
-- descartar la fila. Un resultado sin carrera válida se rechaza, porque

```

```
-- la carrera determina la fecha y el circuito del hecho.
-----
INSERT INTO dim_circuito (sk_circuito, id_circuito_origen, referencia, nombre_circuito,
                        localidad, pais, continente)
VALUES (-1, -1, 'desconocido', 'Desconocido', 'Desconocida', 'Desconocido', 'Desconocido');

INSERT INTO dim_piloto (sk_piloto, id_piloto_origen, referencia, nombre, apellido,
                      nombre_completo, nacionalidad)
VALUES (-1, -1, 'desconocido', 'Desconocido', 'Desconocido', 'Desconocido', 'Desconocida');

INSERT INTO dim_escuderia (sk_escuderia, id_escuderia_origen, referencia, nombre_escuderia,
                          nacionalidad)
VALUES (-1, -1, 'desconocido', 'Desconocida', 'Desconocida');

INSERT INTO dim_estado (sk_estado, id_estado_origen, descripcion_estado, categoria_estado)
VALUES (-1, -1, 'Desconocido', 'Desconocido');

-----
-- 2. Dimensión tiempo: un registro por día
-----
INSERT INTO dim_tiempo (sk_tiempo, fecha, dia, dia_semana, nombre_dia, mes, nombre_mes,
                      trimestre, semestre, anio, decada, nombre_decada)
SELECT TO_CHAR(d, 'YYYYMMDD')::INTEGER,
       d::DATE,
       EXTRACT(DAY FROM d),
       EXTRACT(ISODOW FROM d),
       (ARRAY['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado', 'Domingo'])
       [EXTRACT(ISODOW FROM d)],
       EXTRACT(MONTH FROM d),
       (ARRAY['Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo', 'Junio', 'Julio',
              'Agosto', 'Septiembre', 'Octubre', 'Noviembre', 'Diciembre'])
       [EXTRACT(MONTH FROM d)],
       EXTRACT(QUARTER FROM d),
       CASE WHEN EXTRACT(MONTH FROM d) <= 6 THEN 1 ELSE 2 END,
       EXTRACT(YEAR FROM d),
       EXTRACT(YEAR FROM d)::INTEGER / 10 * 10,
       'Década de ' || (EXTRACT(YEAR FROM d)::INTEGER / 10 * 10)
FROM generate_series(DATE '1950-01-01', DATE '2030-12-31', INTERVAL '1 day') AS g(d);
```

Anexo D · 04_consultas_objetivos.sql

```
-- =====
-- Proyecto Base de Datos 3 - Data Warehouse de Fórmula 1
-- Script 04: consultas analíticas de verificación
-- Cada consulta responde a un objetivo específico (OE) del proyecto y
-- demuestra que el modelo dimensional contiene los datos necesarios.
-- =====

SET search_path TO dw_f1;

-----
-- OE1. Conversión de la posición de salida en resultado final
-- KPI: % de victorias desde la pole y promedio de posiciones ganadas
-----
SELECT c.temperada,
       SUM(f.es_pole) AS largadas_desde_pole,
       SUM(f.es_victoria * f.es_pole) AS victorias_desde_pole,
       ROUND(100.0 * SUM(f.es_victoria * f.es_pole)
             / NULLIF(SUM(f.es_pole), 0), 1) AS pct_conversion_pole,
       ROUND(AVG(f.posiciones_ganadas), 2) AS prom_posiciones_ganadas
FROM fact_resultado_carrera f
JOIN dim_carrera c ON c.sk_carrera = f.sk_carrera
GROUP BY c.temperada
ORDER BY c.temperada;

-----
-- OE2. Dominio de las escuderias por temporada
-- KPI: % de victorias, % de podios y cuota de puntos de la temporada
-----
WITH por_escuderia AS (
  SELECT c.temperada,
         e.nombre_escuderia,
```

```

        SUM(f.es_victoria) AS victorias,
        SUM(f.es_podio)   AS podios,
        SUM(f.puntos)     AS puntos
    FROM fact_resultado_carrera f
    JOIN dim_carrera   c ON c.sk_carrera = f.sk_carrera
    JOIN dim_escuderia e ON e.sk_escuderia = f.sk_escuderia
    GROUP BY c.temporada, e.nombre_escuderia
)
SELECT temporada,
       nombre_escuderia,
       victorias,
       podios,
       puntos,
       ROUND(100.0 * victorias / NULLIF(SUM(victorias) OVER (PARTITION BY temporada), 0), 1) AS pct_victorias,
       ROUND(100.0 * podios / NULLIF(SUM(podios) OVER (PARTITION BY temporada), 0), 1) AS pct_podios,
       ROUND(100.0 * puntos / NULLIF(SUM(puntos) OVER (PARTITION BY temporada), 0), 1) AS cuota_puntos
FROM por_escuderia
ORDER BY temporada, puntos DESC;

-- OE2 (variante). Victorias por escuderia y década (jerarquía de la dimensión tiempo)
SELECT t.nombre_decada,
       e.nombre_escuderia,
       SUM(f.es_victoria) AS victorias
FROM fact_resultado_carrera f
JOIN dim_tiempo   t ON t.sk_tiempo = f.sk_tiempo
JOIN dim_escuderia e ON e.sk_escuderia = f.sk_escuderia
GROUP BY t.decada, t.nombre_decada, e.nombre_escuderia
HAVING SUM(f.es_victoria) > 0
ORDER BY t.decada, victorias DESC;

-----
-- OE3. Confiabilidad de las escuderías
-- KPI: tasa de abandono, tasa de finalización y causas de abandono
-----
SELECT c.temporada,
       e.nombre_escuderia,
       COUNT(*) AS participaciones,
       SUM(f.es_abandono) AS abandonos,
       ROUND(100.0 * SUM(f.es_abandono) / COUNT(*), 1) AS tasa_abandono,
       ROUND(100.0 * SUM(f.es_finalizado) / COUNT(*), 1) AS tasa_finalizacion
FROM fact_resultado_carrera f
JOIN dim_carrera   c ON c.sk_carrera = f.sk_carrera
JOIN dim_escuderia e ON e.sk_escuderia = f.sk_escuderia
GROUP BY c.temporada, e.nombre_escuderia
ORDER BY c.temporada, tasa_abandono DESC;

SELECT s.categoria_estado,
       COUNT(*) AS abandonos,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_del_total
FROM fact_resultado_carrera f
JOIN dim_estado s ON s.sk_estado = f.sk_estado
WHERE f.es_abandono = 1
GROUP BY s.categoria_estado
ORDER BY abandonos DESC;

-----
-- OE4. Rendimiento histórico de los pilotos
-- KPI: puntos por carrera normalizados, tasa de victorias y de podios,
--      edad promedio al ganar
-----
SELECT p.nombre_completo,
       COUNT(*) AS participaciones,
       SUM(f.es_victoria) AS victorias,
       SUM(f.es_podio) AS podios,
       ROUND(SUM(f.puntos_sistema_actual) / COUNT(*), 2) AS puntos_norm_por_carrera,
       ROUND(100.0 * SUM(f.es_victoria) / COUNT(*), 1) AS tasa_victorias,
       ROUND(100.0 * SUM(f.es_podio) / COUNT(*), 1) AS tasa_podios,
       ROUND(AVG(f.edad_piloto) FILTER (WHERE f.es_victoria = 1), 1) AS edad_prom_al_ganar
FROM fact_resultado_carrera f
JOIN dim_piloto p ON p.sk_piloto = f.sk_piloto
GROUP BY p.sk_piloto, p.nombre_completo
HAVING COUNT(*) >= 20 -- mínimo de participaciones para comparar (ajustable)
ORDER BY puntos_norm_por_carrera DESC
LIMIT 20;

```

```

-- OE4 (métrica semiaditiva). Campeón de cada temporada: se toma el valor
-- de puntos_acumulados_temporada en la última ronda; no se suma entre rondas.
SELECT DISTINCT ON (c.temporada)
    c.temporada,
    p.nombre_completo AS campeon,
    f.puntos_acumulados_temporada AS puntos_finales
FROM fact_resultado_carrera f
JOIN dim_carrera c ON c.sk_carrera = f.sk_carrera
JOIN dim_piloto p ON p.sk_piloto = f.sk_piloto
WHERE f.posicion_campeonato = 1
ORDER BY c.temporada, c.ronda DESC;

-- -----
-- OE5. Caracterización de circuitos y países sede
-- KPI: Grandes Premios disputados, índice de adelantamiento,
-- tasa de abandono y % de victorias desde la pole
-- -----
SELECT ci.continente,
    ci.pais,
    ci.nombre_circuito,
    COUNT(DISTINCT f.sk_carrera) AS grandes_premios,
    ROUND(AVG(f.posiciones_ganadas), 2) AS indice_adelantamiento,
    ROUND(100.0 * SUM(f.es_abandono) / COUNT(*), 1) AS tasa_abandono,
    ROUND(100.0 * SUM(f.es_victoria * f.es_pole)
        / NULLIF(SUM(f.es_pole), 0), 1) AS pct_victorias_desde_pole
FROM fact_resultado_carrera f
JOIN dim_circuito ci ON ci.sk_circuito = f.sk_circuito
GROUP BY ci.continente, ci.pais, ci.nombre_circuito
ORDER BY grandes_premios DESC;

-- -----
-- OE6. Evolución del ritmo por circuito (datos desde 2004)
-- KPI: mejor vuelta, velocidad media de vuelta rápida y variación anual
-- -----
WITH ritmo AS (
    SELECT ci.nombre_circuito,
        c.temporada,
        MIN(f.tiempo_vuelta_rapida_ms) AS mejor_vuelta_ms,
        ROUND(AVG(f.velocidad_vuelta_rapida_kmh), 3) AS velocidad_media_kmh
    FROM fact_resultado_carrera f
    JOIN dim_carrera c ON c.sk_carrera = f.sk_carrera
    JOIN dim_circuito ci ON ci.sk_circuito = f.sk_circuito
    WHERE f.tiempo_vuelta_rapida_ms IS NOT NULL
    GROUP BY ci.nombre_circuito, c.temporada
)
SELECT nombre_circuito,
    temporada,
    TO_CHAR(make_interval(secs => mejor_vuelta_ms / 1000.0), 'MI:SS.MS') AS mejor_vuelta,
    velocidad_media_kmh,
    ROUND(100.0 * (velocidad_media_kmh - LAG(velocidad_media_kmh) OVER w)
        / LAG(velocidad_media_kmh) OVER w, 2) AS variacion_pct
FROM ritmo
WINDOW w AS (PARTITION BY nombre_circuito ORDER BY temporada)
ORDER BY nombre_circuito, temporada;

```

Anexo E · 05_pruebas_integridad.sql

```

-- =====
-- Proyecto Base de Datos 3 - Data Warehouse de Fórmula 1
-- Script 05: pruebas de integridad referencial y de dominio
--
-- Inserta un Gran Premio de ejemplo, intenta cinco operaciones inválidas
-- y deshace todo al final (ROLLBACK). Cada operación inválida debe
-- terminar con ERROR. Ejecutar con psql:
-- psql -U postgres -d dw_formula1 -f 05_pruebas_integridad.sql
-- =====

\set ON_ERROR_ROLLBACK on
SET search_path TO dw_f1;

BEGIN;

```

```
-- Datos de ejemplo (se deshacen al final)
INSERT INTO dim_carrera (sk_carrera, id_carrera_origen, temporada, ronda, total_rondas_temporada,
                        nombre_gran_premio, sistema_puntos)
VALUES (1000, 99001, 2023, 1, 22, 'Gran Premio de prueba', '2019-2024: 25-18-15-12-10-8-6-4-2-1 + 1 vuelta rápida');
INSERT INTO dim_circuito (sk_circuito, id_circuito_origen, referencia, nombre_circuito, localidad, pais, continente)
VALUES (1000, 99001, 'prueba', 'Circuito de prueba', 'Localidad', 'País', 'Asia');
INSERT INTO dim_piloto (sk_piloto, id_piloto_origen, referencia, nombre, apellido, nombre_completo, nacionalidad)
VALUES (1000, 99001, 'prueba', 'Piloto', 'Prueba', 'Piloto Prueba', 'Bolivian');
INSERT INTO dim_escuderia (sk_escuderia, id_escuderia_origen, referencia, nombre_escuderia, nacionalidad)
VALUES (1000, 99001, 'prueba', 'Escudería de prueba', 'Bolivian');
INSERT INTO dim_estado (sk_estado, id_estado_origen, descripcion_estado, categoria_estado)
VALUES (1000, 99001, 'Finished', 'Finalizó');
INSERT INTO fact_resultado_carrera (sk_tiempo, sk_carrera, sk_circuito, sk_piloto, sk_escuderia, sk_estado,
                                   id_resultado_origen, codigo_posicion, posicion_salida, posicion_final, orden_llegada,
                                   posiciones_ganadas, puntos, puntos_sistema_actual, vueltas_completadas,
                                   es_victoria, es_podio, es_pole, es_en_puntos, es_finalizado)
VALUES (20230305, 1000, 1000, 1000, 1000, 1000, 99001, '1', 1, 1, 0, 25, 25, 57, 1, 1, 1, 1, 1);

\echo '--- Prueba 1: clave foránea inexistente (sk_piloto = 9999) -> debe fallar'
INSERT INTO fact_resultado_carrera (sk_tiempo, sk_carrera, sk_circuito, sk_piloto, sk_escuderia, sk_estado,
                                   id_resultado_origen, codigo_posicion, posicion_salida, posicion_final, orden_llegada, es_finalizado)
VALUES (20230305, 1000, 1000, 9999, 1000, 1000, 99002, '5', 3, 5, 5, 1);

\echo '--- Prueba 2: indicador incoherente (es_victoria = 1 con posicion_final = 2) -> debe fallar'
INSERT INTO fact_resultado_carrera (sk_tiempo, sk_carrera, sk_circuito, sk_piloto, sk_escuderia, sk_estado,
                                   id_resultado_origen, codigo_posicion, posicion_salida, posicion_final, orden_llegada,
                                   es_victoria, es_podio, es_finalizado)
VALUES (20230305, 1000, 1000, 1000, 1000, 1000, 99003, '2', 3, 2, 2, 1, 1, 1);

\echo '--- Prueba 3: resultado duplicado (id_resultado_origen repetido) -> debe fallar'
INSERT INTO fact_resultado_carrera (sk_tiempo, sk_carrera, sk_circuito, sk_piloto, sk_escuderia, sk_estado,
                                   id_resultado_origen, codigo_posicion, posicion_salida, orden_llegada, es_abandono)
VALUES (20230305, 1000, 1000, 1000, 1000, 1000, 99001, 'R', 3, 6, 1);

\echo '--- Prueba 4: borrar un piloto que tiene hechos asociados -> debe fallar'
DELETE FROM dim_piloto WHERE sk_piloto = 1000;

\echo '--- Prueba 5: puntos negativos -> debe fallar'
INSERT INTO fact_resultado_carrera (sk_tiempo, sk_carrera, sk_circuito, sk_piloto, sk_escuderia, sk_estado,
                                   id_resultado_origen, codigo_posicion, posicion_salida, posicion_final, orden_llegada, puntos, es_finalizado)
VALUES (20230305, 1000, 1000, 1000, 1000, 1000, 99005, '12', 3, 12, 12, -2, 1);

\echo '--- Control: la tabla de hechos conserva solo el registro válido de ejemplo'
SELECT id_resultado_origen, codigo_posicion, puntos, es_victoria
FROM fact_resultado_carrera
WHERE id_resultado_origen BETWEEN 99001 AND 99005;

ROLLBACK;
```